Linear Programming Approach to the Graph Square Problem

Eddie Xu

School of Mathematics and Statistics
University of Melbourne

Submitted for the degree of Master of Science

Supervisor: Dr Christopher Duffy

October 13, 2026

Abstract

The square of a graph is obtained by joining every pair of vertices whose distance is at most two. Given a graph G, the graph square root problem asks whether there exists a graph H such that $G = H^2$. Whilst this problem admits polynomial-time algorithms for restricted classes of root graphs, the general decision problem is **NP**-complete.

In this thesis, rather than studying the construction problem directly and using reduction techniques to ascertain polynomial-time algorithms, we formulate the problem as an integer program based on the relationship between the adjacency matrices of a root H and its square G. By constructing the corresponding linear relaxation, the problem can then be situated within polynomial time through the use of interior-point methods.

The seminal contribution is that of Ross and Harary, who approached the problem by restricting the root graphs to trees. By analyzing how many cliques each vertex of the graph square lies in, they gave an algorithm for finding the unique tree root up to isomorphism. In contrast, our formulation provides an optimization-based approach that can be iteratively strengthened through the addition of further constraints. These constraints refine the linear relaxation, enabling tree roots to be represented by integral solutions to the linear program, while graphs with no possible roots are identified through fractional or infeasible solutions. Rather than relying purely on heuristics, our approach exploits structural properties of the graph square and its root to obtain an exact solution.

Acknowledgements

I would like to thank my supervisor Christopher Duffy for helping and introducing me to this wonderful topic. Although this exploration has somewhat deviated from your vision, I greatly appreciate your willingness to lend your knowledge to support me in writing this thesis.

I would also like to thank my family and friends for their support and help, without which I would not have been able to do this.

Contents

1 Introduction **3**
- 1.1 Background . . . 3
- 1.2 Literature . . . 4
  - 1.2.1 Overview . . . 5
- 1.3 Definitions and Notations . . . 6
  - 1.3.1 Complexity Theory . . . 6
  - 1.3.2 Graph Theory . . . 8
  - 1.3.3 Mathematical Optimization . . . 11

2 Algorithmic Approaches **14**
- 2.1 NP-Completeness Proof . . . 14
  - 2.1.1 NP-Hard proof . . . 15
- 2.2 Algorithm for Tree Roots . . . 19
- 2.3 Mathematical Optimization Approach . . . 23
  - 2.3.1 Formulation . . . 25
  - 2.3.2 Validity of $\mathcal{IP}(G)$ . . . 27

3 Square of a Tree using Linear Programming **30**
- 3.1 Linear Program Relaxation . . . 31
- 3.2 Min Linear Program . . . 33
- 3.3 Max Linear Program . . . 35
  - 3.3.1 Fundamental Properties . . . 36
- 3.4 Leaf Fixing . . . 38
  - 3.4.1 Single Triangular Decomposition . . . 39
  - 3.4.2 Tetrahedron Decomposition . . . 39
- 3.5 Unicliqual Vertex Linear Program . . . 41
- 3.6 Tree Proof . . . 42

4 Further Work **45**
- 4.1 Vertex Deletion . . . 45
- 4.2 Symbolic Sets . . . 48
- 4.3 Duality . . . 50
- 4.4 Rounding . . . 54

Chapter 1

Introduction

1.1 Background

In 1949, Leon Festinger proposed a novel method in the field of social psychology for analyzing relationships among a number of people present [FBB23]. Classically, this was represented using a sociogram, a visual tool that maps out the relationships between people. However, Forsyth and Katz introduced the use of a matrix to represent the sociometric pattern, with rows and columns corresponding to individuals, 1s for choices, and blank diagonals. Festinger noticed that his structure, reminiscent of the adjacency matrix defined by König in [Kön36], also had the remarkable property that the action of squaring a matrix

$$A_{rc}^2 = A_{1c}A_{r1} + \cdots + A_{nc}A_{rn}$$ [1]

represented the number of "two-step" connections. This same approach can then be used to denote "three-step" connections between any two people

$$A_{rc}^3 = A_{1c}A_{r1}^2 + \cdots + A_{nc}A_{rn}^2,$$ [2]

Higher steps can be easily generated in the same way. In this manner, higher powers of the adjacency matrix encode the number of walks of increasing length between pairs of individuals. For the two-step case, this allows densely connected groups of people, known as cliques, to be identified.

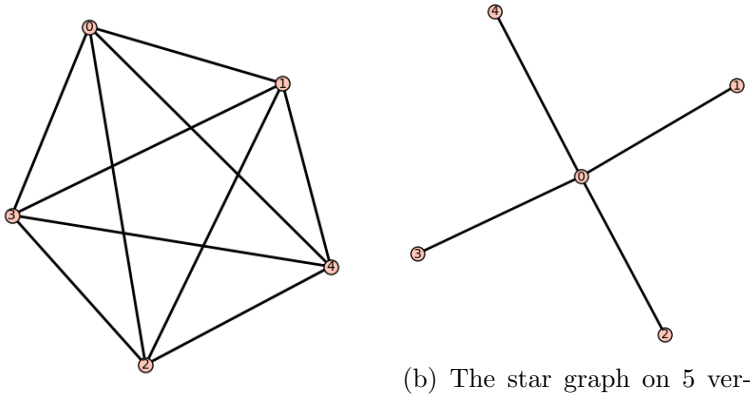
In the process of improving Festinger's clique detection methods, Ross and Harary showed that complete graphs may be realized as squares of stars by considering powers of such matrices [RH60]. They introduced a structural characterization of the square of a tree in terms of clique intersections, "multicliqual" vertices, and so called "neighborly" properties, proving that a graph is the square of a tree if and only if

i) every vertex is every vertex is adjacent to at least one other vertex,

ii) no two maximal cliques intersect in more than two vertices, and

[1] A_{rc}^2 is more naturally written as $\sum_{k=1}^{n} A_{ck}A_{kr}$ using the commutativity of multiplication $A_{1c}A_{r1} = A_{c1}A_{1r}$, which intuitively gives the notion of paths from c to r via intermediary k

[2] A_{rc}^3 is more naturally written as $\sum_{k=1}^{n} A_{ck}A_{kr}^2$, which intuitively counts the number of paths from c to r that go through an intermediate vertex k, followed by any two-step path from k to r.



(a) The clique on 5 vertices K_5.

(b) The star graph on 5 vertices $K_{1,4}$ One of the 15 non-isomorphic graph roots of K_5.

Figure 1.1: K_5 and two of its 15 non-isomorphic graph roots.

iii) the number of maximal clique pairs intersecting in two vertices is exactly one less than the total number of maximal cliques.

Using these properties, they constructed an explicit algorithm that recovers the underlying tree for a given graph square. Their work has become the foundational reference for the study of graph squares and is universally cited in subsequent literature on the subject, and will be explored in more detail later.

Since then, the problem of finding the root of a graph has extended beyond sociology and is a well-studied field in both mathematics and computer-science, with direct applications in distributed computing (e.g., see [PA23]), network design and computational biology (e.g., see [Zho+25]). This is partly because many structural properties of graph roots are first established in the square case before being extended to the more general k-th power setting (e.g., see [HKT67]).

Research in this area has primarily focused around an undirected graph H and its square $G = H^2$, which is obtained by adding an edge between every pair of vertices whose distance in H is at most two. This two-step undirected graph square has been studied extensively with much of the focus being on the complexity of the decision problem - given a specific graph square G does there exist a root graph H such that $H^2 = G$. Motwani and Sudan showed that deciding if a graph admits a root is **NP**-complete. Subsequent studies in the area have restricted the decision problem in two ways, either by considering various properties on the root graph or by considering properties on the graph square.

1.2 Literature

Focusing first on special undirected graph classes, the aforementioned work by Ross and Harary in 1960 gives an algorithm for constructing a tree root by using characterizations of the graph square. As part of this work they proved tree roots are unique up to isomorphism. This was the first major contribution to the study of graph roots and provided fundamental structural characteristics of graph squares, laying the groundwork for later research on graph root problems and their applications. Building upon these insights, Lin and Skiena bypassed the inefficiency of general maximal clique enumeration in Ross and Harary's approach, by developing an optimal $\mathcal{O}(|V| + |E|)$ linear time algorithm to recognize tree squares and reconstruct their unique tree roots[LS95].

Fleischner in 1974 showed that as long as a graph is 2-connected its graph square will have a Hamiltonian cycle. This established a broad condition guaranteeing Hamiltonicity in graph squares which is used in many subsequent works such as [BMR14].

Lau and Corneil in 2004 provided an algorithm for constructing roots from an interval square in polynomial time, and showed that the decision problem for finding roots of chordal graphs, roots of split graphs, and chordal roots of graphs are **NP**-complete [LC04]. Furthermore, Lau gave a linear-time algorithm for constructing bipartite roots, and proved that deciding whether a bipartite square has a cube root is **NP**-complete [Lau06]. Le and Tuy in 2007 generalized tree root results further by giving a linear time recognition of roots of block graphs [LT07].

Generalizing the notion of graph squares, in 1967 Mukhopadhyay provided necessary and sufficient conditions for a connected graph to have a square root [Muk67], whilst Geller in 1970 likewise found necessary and sufficient conditions for a digraph to have a square root [Gel70].

In 1994 Motwani and Sudan established the complexity of the square problem for the generalized undirected case and showed that the problem of determining if a given graph has a square root is an **NP**-complete problem via a polynomial reduction to 3-CNF not-all-equal SAT [MS94].

Their paper led to joint work by Farzad, Le, Tuy, Lau and Karimi in 2009 and 2012, in which they established a partial dichotomy theorem for the complexity of determining if a given undirected graph of girth g has a square root [Far+09] which was informally completed under [FK12]. In 2013, Cochefert, Couturier, Golovach, Kratsch, and Paulusma gave an algorithm for constructing a square root with minimal number of edges from undirected graphs with max degree ≤ 6 in polynomial time and an algorithm for constructing maximal edge square roots in exponential time [Coc+13].

In recent years, Duffy provided a full classification for the undirected square of oriented graphs satisfying max degree ≤ 3 and girth ≥ 4 [Duf25].

1.2.1 Overview

Having surveyed the relevant literature, we now turn to the foundations needed for the remainder of the thesis. We first standardize the definitions that will be used for the rest of this thesis. We then introduce fundamental theory on the three areas of: complexity theory, graph theory and mathematical optimization, which are used in the various algorithmic approaches to the decision problem.

The first construction we examine is the reduction used by Motwani and Sudan in showing that the decision problem determining whether a given graph G admits a square root is **NP**-complete [MS94]. Next, we introduce the algorithm given by Ross and Harary for constructing a tree root from a given square in detail. Finally, we formulate the graph square problem as an integer program.

In Chapter 3 we present the ideas of linear relaxation and discuss the results of each program and their validity. To obtain these results, we conducted an exhaustive enumeration on all non-isomorphic connected graphs up to $n = 8$ vertices in addition to obtaining theoretical results. For each such graph, we formulate the corresponding linear program and solved it using a standard LP solver using interior point method. The outcomes of these computations are categorized as integral, fractional, or infeasible and are analyzed.

Next, we introduce several augmentations to the linear program, consisting of constraints that encode structural bounds on both the graph square and its underlying root. In particular, we show that by adding specific constraints to the LP, we can ensure that the square of all trees will return a feasible integer solution. This augmentation ensures that the LP returns a tree root in polynomial time, whilst ensuring that all squares with roots remain LP feasible.

Finally, in Chapter 4, we leverage the augmented linear program together with specific structural properties of graph squares to develop two algorithms that iteratively solve modified linear programs

with additional constraints. Depending on the outcome, these algorithms proceed either greedily or exhaustively, with the choice guided by whether the resulting search remains polynomial. The symbolic group algorithm exploits the non-existence of an edge in a triangle K_3, while the vertex deletion algorithm relies on finding root solutions for subgraphs of the square. We then examine duality, one of the more powerful tools provided by linear programming, before presenting a simple yet effective rounding heuristic.

1.3 Definitions and Notations

1.3.1 Complexity Theory

Complexity theory studies the classification of computational problems and explores the amount of resources required to solve these problems. Complexity theory considers both time and space as critical resources and provides a standardized approach across various different problems.

Definition 1.3.1 (Big-O Notation)**.** *Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. The set $O(g(n))$ is defined as*

$$O(g(n)) = \{f \mid \text{ there exists } c > 0, n_0 \in \mathbb{N} \text{ such that } f(n) \leq cg(n) \text{ for all } n \geq n_0\}.$$

We write $f(n) = O(g(n))$ as shorthand for $f(n) \in O(g(n))$.

Using this notation, we can classify algorithms based on their runtime.

Definition 1.3.2 (Polynomial-Time Algorithm)**.** *An algorithm is said to run in **polynomial time** if there exists a constant $c \geq 0$ such that its run time on inputs of size n is $O(n^c)$.*

Polynomial-time algorithms are important because they are generally regarded as efficient and computationally "simple". Consequently, the class of problems that admit polynomial-time algorithms forms the basis for the complexity class **P**.

Definition 1.3.3 (Decision Problem)**.** *A **decision problem** is a computational problem whose output is either **Yes** or **No** for every valid input instance.*

Definition 1.3.4 (Class **P**)**.** *The complexity class* **P** *consists of all decision problems that can be solved by a deterministic algorithm in polynomial time.*

Unfortunately many important decision problems are not known to admit such polynomial-time algorithms and hence need additional classification on their "hardness". For example, determining whether a graph is 3-colourable is a well known problem for which no polynomial-time algorithm is known, despite the fact that a proposed 3-colouring can be checked efficiently. This motivates the study of complexity classes based not only on how efficiently a solution can be found, but also on how a proposed solution can be verified efficiently.

Definition 1.3.5 (Certificate)**.** *Let L be a decision problem. A **certificate** for an instance x of L is a finite string c that provides evidence that x is a **Yes**-instance of L. A certificate is said to be **polynomially verifiable** if there exists a deterministic algorithm that for given input (x, c), can determine whether c is a valid certificate for x in time polynomial in $|x|$.*

Definition 1.3.6 (Class **NP**)**.** *The complexity class* **NP** *consists of all decision problems for which every **Yes**-instance admits a certificate that is polynomially verifiable.*

Since every decision problem in **P** can be solved in polynomial time, a solution can also be verified in polynomial time,

$$\mathbf{P} \subseteq \mathbf{NP}.$$

Whether this inclusion is strict remains one of the most important open questions in theoretical computer science - the **P** vs **NP** problem. Polynomial-time reductions provide a means of transforming instances of one problem into instances of another while preserving their **Yes**/**No** status. In particular, if a problem in **NP** can be reduced in polynomial time to a problem that is solvable in polynomial time, then the original problem is also solvable in polynomial time. If $\mathbf{P} = \mathbf{NP}$, this would imply that every problem in **NP** is solvable in polynomial time.

Definition 1.3.7 (Polynomial-Time Reduction)**.** *Let A and B be decision problems, with sets of instances $\mathcal{I}_A$ and $\mathcal{I}_B$ and sets of **Yes**-instances $Y_A \subseteq \mathcal{I}_A$ and $Y_B \subseteq \mathcal{I}_B$. A **polynomial-time reduction** from A to B, denoted $A \leq_p B$, is a polynomial-time computable function $f\colon \mathcal{I}_A \to \mathcal{I}_B$ such that, for every instance $x \in \mathcal{I}_A$,*

$$x \in Y_A \quad \textit{if and only if} \quad f(x) \in Y_B.$$

This notion of reduction allows the computational difficulty of problems to be compared. In particular, a problem is **NP**-hard if every problem in **NP** can be reduced to it via a polynomial time reduction.

Definition 1.3.8 (Class **NP**-Hard)**.** *A decision problem B is **NP-hard** if every problem $A \in \mathbf{NP}$ satisfies $A \leq_p B$. The collection of all **NP**-hard problems is denoted by the class **NP-hard**.*

Importantly, an **NP**-hard problem may not necessary lie in **NP**. If it does, it is called **NP**-complete.

Definition 1.3.9 (Class **NP**-Complete)**.** *A decision problem B is **NP-complete** if $B \in \mathbf{NP}$, and B is **NP**-hard. The collection of all **NP**-complete problems is denoted by the class **NP-complete**.*

One can establish that a problem is **NP**-complete by showing that it belongs to **NP** and that a known **NP**-complete problem can be reduced to it in polynomial time.

The first known **NP**-complete problem was the Boolean satisfiability problem, or SAT, as established by the Cook-Levin theorem [Coo71].

Definition 1.3.10 (Boolean Satisfiability Problem)**.** *The **Boolean satisfiability problem (SAT)** asks whether there exists an assignment of truth values to the variables of a given Boolean formula such that the formula evaluates to* TRUE*. A Boolean formula is said to be **satisfiable** if such an assignment exists.*

Any Boolean formula can be transformed into an equisatisfiable conjunctive normal form (CNF) formula of linear size by introducing additional variables using the **Tseitin transformation** [Tse68]. This transformation preserves satisfiability while ensuring only linear growth in the size of the resulting conjunctive normal form Boolean formula.

Definition 1.3.11 (Conjunctive Normal Form)**.** *A Boolean formula is in **conjunctive normal form (CNF)** if it is expressed as a conjunction of clauses, where each clause is a disjunction of literals. That is, a CNF formula has the form*

$$C_1 \wedge C_2 \wedge \cdots \wedge C_m, \quad \textit{where each clause } C_i \textit{ is of the form} \quad C_i = \ell_{i1} \vee \ell_{i2} \vee \cdots \vee \ell_{ik_i}$$

and each ℓ_{ij} is either a Boolean variable or its complement, denoted with $\overline{\ell_{ij}}$.

A further restriction is obtained by requiring each clause in a CNF formula to contain exactly three literals. This can be done using 3-CNF transformation and gives the following problem.

Definition 1.3.12 (3-SAT)**.** *The* 3-***SAT*** *problem asks whether there exists an assignment of truth values to the variables of a Boolean formula in conjunctive normal form such that every clause contains exactly three literals and the formula evaluates to* TRUE*. It is formally structured as*

Instance: *A Boolean formula φ in conjunctive normal form in which each clause contains exactly three literals.*

Question: *Does there exist an assignment of truth values to the variables of φ such that every clause is satisfied?*

The decision problem **3-SAT** is a well-known **NP**-complete problem. It is commonly used as a starting point for proving the **NP**-completeness of other decision problems. In particular, we will reduce 3-SAT to the **SQUARE** [3] problem to establish that the latter is **NP**-complete.

1.3.2 Graph Theory

Informally, a graph is a mathematical structure used to represent relationships between a collection of discrete objects.

Definition 1.3.13 (Graph)**.** *A* ***graph*** *G is an ordered pair (V, E), where V is a set of vertices and E is a set of unordered pairs of distinct vertices in V. Each element of E is called an* ***edge*** *of G.*

The number of vertices and edges in a graph provide basic measures of its order and size.

Definition 1.3.14 (Order and Size)**.** *The* ***order*** *of a graph G, denoted by $|G|$, is the number of vertices in G, while the* ***size*** *of G is the number of edges. Thus, if $V_G = \{v_1, \ldots, v_n\}$ and $E_G = \{e_1, \ldots, e_m\}$, then $|G| = n$ and $|E_G| = m$.*

For a vertex v, the number of edges incident to it is its degree.

Definition 1.3.15 (Degree)**.** *The* ***degree*** *of a vertex v in a graph G, denoted by $\deg_G(v)$, is the number of edges incident to v. The minimum and maximum degrees of G are denoted by*

$$\delta(G) = \min_{v \in V_G} \deg_G(v) \quad \textit{and} \quad \Delta(G) = \max_{v \in V_G} \deg_G(v),$$

respectively.

The degree of a vertex can also be described through the vertices to which it is adjacent to.

Definition 1.3.16 (Neighbourhood)**.** *For a vertex $v \in V_G$, the* ***open neighbourhood*** *of v denoted by $N_G(v)$, is the set of vertices adjacent to v in G. The* ***closed neighbourhood*** *of v includes vertex v and is denoted by*

$$N_G[v] = N_G(v) \cup \{v\}.$$

Connections between vertices in a graph can be described using walks, paths and cycles.

Definition 1.3.17 (Walk)**.** *A* ***walk*** *from a vertex v_i to a vertex v_j in a graph G is an alternating sequence of vertices and edges*

$$W = \langle v_i, e_1, v_1, e_2, \ldots, e_k, v_j \rangle,$$

where each edge e_r has endpoints given by the vertices immediately preceding and following it in the sequence. The number of edges in W is called the ***length*** *of the walk.*

[3] The **SQUARE** problem is defined in Definition 2.1.1.

Definition 1.3.18 (Path)**.** *A **path** is a walk in which no vertex is repeated. A path containing p vertices is denoted by P_p.*

Definition 1.3.19 (Cycle)**.** *A **cycle** is a closed path in which the first and last vertices are the same and all other vertices are distinct. A cycle containing c vertices is denoted by C_c.*

The length of a shortest cycle is an important structural parameter of a graph.

Definition 1.3.20 (Girth)**.** *The **girth** of a graph G, denoted by $g(G)$, is the length of the shortest cycle in G. If G contains no cycle, its girth is ∞.*

A finite connected graph with infinite girth is called a tree.

Definition 1.3.21 (Tree)**.** *A **tree** is a connected graph that contains no cycles.*

The vertices of a tree can be classified according to their degree.

Definition 1.3.22 (Leaf)**.** *A **leaf** of a tree T is a vertex of degree one.*

Definition 1.3.23 (Stem)**.** *A **stem** of a tree T is the vertex adjacent to a leaf of T.*

The length of a shortest path provides a natural measure of the distance between two vertices.

Definition 1.3.24 (Distance)**.** *The **distance** between two vertices u and v on the graph G, denoted by $d_G(u, v)$, is the number of edges of the shortest path between vertices u and v in G. If no such path exists, $d_G(u, v) = \infty$.*

Definition 1.3.25 (Diameter)**.** *The **diameter** of a graph G is*

$$\operatorname{diam}(G) = \max_{u,v \in V_G} d_G(u, v).$$

For a disconnected graph, the diameter is ∞.

If a graph has finite diameter it is connected.

Definition 1.3.26 (Connected Graph)**.** *A graph G is **connected** if there exists a path between every pair of vertices in G Otherwise, G is **disconnected**.*

A graph can be further categorized according to whether multiple edges between a vertex and itself are permitted, whether there are infinite number of vertices and edges, whether the edges represent an unordered or ordered relationship between their endpoints, and whether the vertices are labelled or unlabelled.

Definition 1.3.27 (Simple Graph)**.** *A **simple graph** $G = (V, E)$ is a graph in which no two distinct edges connect the same pair of vertices and no edge connects a vertex to itself.*

Definition 1.3.28 (Finite Graph)**.** *A **finite graph** is a graph $G = (V, E)$ for which both V and E are finite sets. A graph that is not finite is called an **infinite graph**.*

Definition 1.3.29 (Directed Graph)**.** *A **directed graph** $G = (V, E)$ is a graph in which each edge $e \in E$ is an ordered pair of distinct vertices in V. Thus, (u, v) represents an edge directed from u to v, where (u, v) may not necessarily be equal to (v, u).*

*An **undirected graph** $G = (V, E)$ is a graph in which each edge $e \in E$ is an unordered pair of distinct vertices in V. Thus, $\{u, v\} = \{v, u\}$ for every edge between vertices u and v.*

Definition 1.3.30 (Labelled Graph)**.** *A **labelled graph** is a graph $G = (V, E)$ together with a labelling function $\ell : V \to L$, where L is a set of labels. A graph is **unlabelled** when no such vertex labels are specified.*

In this thesis, we exclusively study finite, simple, undirected, unlabelled graphs. However, the constructions and algorithms may use additional structure, including labels or multiple edges, which must be analyzed and restricted to their corresponding simple, unlabelled counterparts.

In particular, two graphs may have different vertex sets while nevertheless having the same underlying structure. This motivates the following notion of isomorphism.

Definition 1.3.31 (Graph Isomorphism)**.** *Let G and H be graphs. An **isomorphism** from G to H is a bijection*

$$f : V_G \to V_H \quad \text{such that} \quad uv \in E_G \text{ if and only if } f(u)f(v) \in E_H.$$

*We say that G is **isomorphic** to H, written $G \cong H$, if there exists an isomorphism from G to H.*

The isomorphism relation is an equivalence relation on the set of graphs, and an **isomorphism class** is an equivalence class of graphs under this relation. Thus, graphs within the same isomorphism class have the same structure up to a relabelling of their vertices.

We will also consider graphs obtained by restricting the vertices and edges of another graph.

Definition 1.3.32 (Subgraph)**.** *A graph $H = (V_H, E_H)$ is a **subgraph** of a graph $G = (V_G, E_G)$ if*

$$V_H \subseteq V_G \quad \text{and} \quad E_H \subseteq E_G.$$

*For $S \subseteq V_G$, the **induced subgraph** of G on S, denoted by $G[S]$, is the graph with vertex set S and edge set*

$$E_{G[S]} = \{uv \in E_G : u, v \in S\}.$$

One of the most important types of graphs is the complete graph, and its subgraph counterpart.

Definition 1.3.33 (Complete Graph)**.** *A graph on n vertices is **complete** if every pair of distinct vertices is adjacent. The complete graph on n vertices is denoted by K_n.*

Definition 1.3.34 (Clique)**.** *A **clique** of a graph G is a subset $S \subseteq V_G$ such that the induced subgraph $G[S]$ is complete. S is a **maximal clique** if there is no clique $S' \subseteq V_G$ such that $S \subsetneq S'$. The **clique number** of G, denoted by $\omega(G)$, is the size of a largest clique in G. A clique containing k vertices is denoted by K_k.*

It is important to distinguish maximal cliques from cliques in general since any vertex belonging to a clique of size n naturally belongs to a clique of size $n - 1$.

Definition 1.3.35. *A **multicliqual vertex** is a vertex that belongs to multiple maximal cliques. A **unicliqual vertex** is a vertex that belongs to only one maximal clique.*

In addition to being specified by a set of vertices and a set of edges, a graph can also be represented using a matrix.

Definition 1.3.36 (Adjacency Matrix)**.** *The **adjacency matrix** of graph H is denoted by $A_H \in \{0, 1\}^{n \times n}$ where*

$$(A_H)_{ij} = \begin{cases} 1, & \text{if } ij \in E_H, \\ 0, & \text{otherwise,} \end{cases}$$

The degrees of the vertices can similarly be represented by a diagonal matrix.

Definition 1.3.37 (Degree Matrix)**.** *The **degree matrix** of graph H is denoted by $D_H \in \mathbb{R}^{n\times n}$ where*

$$(D_H)_{ij} = \begin{cases} \deg_H(i), & \text{if } i = j, \\ 0, & \text{otherwise.} \end{cases}$$

In our discussion of graphs represented as matrices, we will find the following notation useful.

Definition 1.3.38 (Indicator Function)**.** *Let $\mathbb{1}_{\geq 1} : \mathbb{R}^{n\times n} \to \{0,1\}^{n\times n}$ denote the entrywise **indicator function**, defined by*

$$\mathbb{1}_{\geq 1}(M)_{ij} = \begin{cases} 1, & \text{if } M_{ij} \geq 1, \\ 0, & \text{otherwise.} \end{cases}$$

We now introduce the graph-theoretic objects central to this thesis.

Definition 1.3.39 (Graph Root and the Square of a Graph)**.** *Let the **graph root** $H = (V, E_H)$ be a finite, simple, undirected graph, where V is the vertex set with $|V| = n$, and $E_H \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}$ is the edge set. The **square of a graph** H, denoted by $H^2 = (V, E_{H^2})$ is the graph on the same vertex set with edge set*

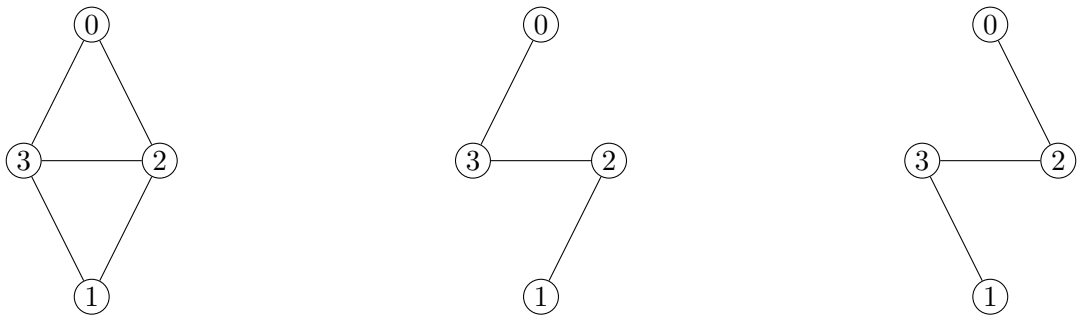
$$E_{H^2} = \{uv \mid u \neq v, 1 \leq d_H(u, v) \leq 2\}. \tag{1.1}$$

*That is, two distinct vertices are adjacent in H^2 if and only if they are connected by a path of length one or two in H. When $G = H^2$ we say that H is a **root** of G*

Equivalently, G is a root of H when the adjacency matrices of G and H satisfy

$$A_G = \mathbb{1}_{\geq 1}\big((A_H)^2 - D_H + A_H\big). \tag{1.2}$$

We prove this equivalence in Chapter 2.



(a) G (b) T_1 (c) T_2

Figure 1.2: The graph G has two possible P_4 square roots that are trees, T_1 and T_2. The two roots are isomorphic, differing only by the interchange of vertices 2 and 3.

1.3.3 Mathematical Optimization

Mathematical optimization studies the problem of finding the best solution from all feasible solutions which are defined by a collection of constraints. An optimization problem usually consists of decision variables, an objective function to be maximized or minimized, and a set of constraints restricting the feasible values of the variables.

Definition 1.3.40 (Integer Program)**.** *An **integer program** (IP) is an optimization problem of the form*

$$\begin{aligned} \textit{max/min} \quad & c^\top x \\ \textit{subject to} \quad & Ax \leq b, \\ & x \in \mathbb{Z}^n, \end{aligned}$$

where $x \in \mathbb{Z}^n$ is the vector of decision variables, $c \in \mathbb{R}^n$ is the objective coefficient vector, $A \in \mathbb{R}^{m \times n}$ is the constraint matrix, and $b \in \mathbb{R}^m$ is the constraint vector.

A linear relaxation is obtained by removing the integrality requirements from an integer program while retaining its linear objective function and constraints.

Definition 1.3.41 (Linear Relaxation)**.** *Let IP be an integer program. The **linear relaxation** of IP is the linear program obtained by removing the requirement that its variables take integer values.*

Definition 1.3.42 (Linear Program)**.** *A **linear program** (LP) is an optimization problem of the form*

$$\begin{aligned} \textit{max/min} \quad & c^\top x \\ \textit{subject to} \quad & Ax \leq b, \\ & x \in \mathbb{R}^n, \end{aligned}$$

where $x \in \mathbb{R}^n$ is the vector of decision variables, $c \in \mathbb{R}^n$ is the objective coefficient vector, $A \in \mathbb{R}^{m \times n}$ is the constraint matrix, and $b \in \mathbb{R}^m$ is the constraint vector.

Definition 1.3.43 (Feasible Solution)**.** *A **feasible solution** to an optimization problem is an assignment of values to its decision variables that satisfies all of its constraints. The set of all feasible solutions is called the **feasible set** and is denoted by $\mathcal{F}$. Geometrically, $\mathcal{F}$ is the subset of the decision-variable space satisfying all of the constraints and is referred to as the **feasible region**.*

Among the feasible solutions, an optimal solution is one that achieves the optimum value of the objective function.

Definition 1.3.44 (Optimal Solution)**.** *An **optimal solution** is a feasible solution that maximizes or minimizes the objective function, depending on the optimization problem. The corresponding value of the objective function is called the **optimal value** and is denoted by* OPT*.*

When an integer program is relaxed to a linear program, every feasible integer solution remains feasible for the relaxation. Consequently, if $\mathcal{F}_{IP}$ and $\mathcal{F}_{LP}$ denote the feasible sets of the integer program and its linear relaxation respectively, then

$$\mathcal{F}_{IP} \subseteq \mathcal{F}_{LP}.$$

Geometrically, $\mathcal{F}_{IP}$ is represented by the feasible integer points in the decision-variable space, whereas $\mathcal{F}_{LP}$ is the polyhedron defined by the relaxed constraints and may contain additional points.

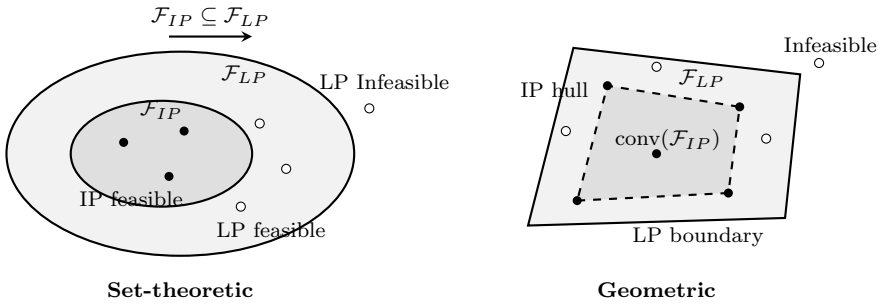


Figure 1.3: Set-theoretic and geometric relationships between an integer program and its linear relaxation.

For a maximization problem, this set inclusion implies $\text{OPT}_{IP} \leq \text{OPT}_{LP}$. Therefore the optimal value for the linear relaxation provides an upper bound on the optimal value of the integer program. For a minimization problem, $\text{OPT}_{LP} \leq \text{OPT}_{IP}$, so the linear relaxation provides a lower bound.

Definition 1.3.45 (Integer Hull)**.** *Let $\mathcal{F}_{IP}$ denote the feasible set of an integer program. The **integer hull** is the convex hull of the feasible integer solutions, denoted by* $\text{conv}(\mathcal{F}_{IP})$*. That is, the integer hull is the set of all convex combinations of points in $\mathcal{F}_{IP}$.*

Since the feasible region of a linear program is convex, the integer hull is contained in the feasible region of its linear relaxation. Hence,

$$\mathcal{F}_{IP} \subseteq \text{conv}(\mathcal{F}_{IP}) \subseteq \mathcal{F}_{LP}.$$

The integer hull is the smallest convex set containing all feasible integer solutions. Thus, geometrically, the gap between the integer hull and the LP feasible region represents the additional solutions admitted by the relaxation. If

$$\text{conv}(\mathcal{F}_{IP}) = \mathcal{F}_{LP},$$

then the linear relaxation coincides with the integer hull and is therefore an exact convex description of the feasible integer solutions.

Chapter 2

Algorithmic Approaches

In this chapter, we explore different methodologies for studying properties of graph roots and graph squares. These include polynomial-time reductions for establishing **NP**-completeness, structural characterisations and constructive algorithms for restricted cases, and integer programming formulations for studying graph roots.

2.1 NP-Completeness Proof

We now establish the computational complexity of the graph square problem as was done by Motwani and Sudan in their paper [MS94]. The decision version of the problem asks whether a given graph admits a root.

Problem 2.1.1 (SQUARE)**.**
Instance: *A finite, simple, undirected graph $G = (V, E)$.*

Question: *Does there exist a finite, simple, undirected graph H such that*

$$G = H^2?$$

Theorem 1 (**NP**)**.** *The decision problem **SQUARE** is in* **NP**.

NP Proof. Given a candidate square root H on n vertices, it is sufficient to verify that $G = H^2$. For each pair of distinct vertices $u, v \in V_H$, check whether u and v are adjacent in H or have a common neighbour in H. This determines whether $d_H(u, v) \leq 2$ and therefore whether $uv \in E_{H^2}$.

There are $\binom{n}{2} = \frac{n(n-1)}{2}$ unordered pairs of vertices. For each pair, determining whether the vertices have a common neighbour requires at most $O(n)$ operations. Hence, the total verification time is $O(n^2) \cdot O(n) = O(n^3)$. By the definition of Big-O notation, this is polynomial in n. Therefore, the certificate H can be verified in polynomial time, and consequently

$$\text{SQUARE} \in \mathbf{NP}.$$

□

Problem 2.1.2 (NAESAT)**.**
Instance: *A 3-CNF formula*

$$\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$$

where each clause C_i is over variables $x_1, \ldots, x_n$.

Question: *Does there exist an assignment of* TRUE *and* FALSE *to the variables such that every clause contains at least one* TRUE *literal and at least one* FALSE *literal?*

Thus, an assignment satisfies the not-all-equal condition if, for every clause, its literals are not all assigned the same truth value. The problem NAESAT is known to be **NP**-complete by Schaefer's dichotomy theorem [Sch78].

2.1.1 NP-Hard proof

Following [MS94], we show **SQUARE** is **NP**-hard by giving a polynomial-time reduction from **NP**-complete NAESAT and then show that G_ϕ has a root if and only if $\phi \in$ NAESAT.

Polynomial Reduction. Let $\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$ be an instance of NAESAT, where ϕ is a 3-CNF formula on variables $x_1, \ldots, x_n$. Over variables $x_1, \ldots, x_n$, construct a graph $G_\phi = (V_\phi, E_\phi)$ such that

$$\phi \in \text{NAESAT} \quad \text{if and only if} \quad G_\phi \text{ has a square root.}$$

The vertex set V_ϕ consists of

(i) Two constant vertices TRUE and FALSE, plus a vertex x and three tail vertices t_1, t_2, t_3 for x.

(ii) For each of the $2n$ literals $\ell_1, \ldots, \ell_{2n}$, where $\ell_{i+n} = \overline{\ell_i}$, a literal vertex L_i.

(iii) For every pair of literals ℓ_i, ℓ_k that are not complements of each other, a literal-pair vertex A_{ik}.

(iv) For each clause $C_j = \{\ell_{j_1}, \ell_{j_2}, \ell_{j_3}\}$, a clause vertex C_j together with three tail vertices C_j^1, C_j^2, C_j^3.

The edge set E_ϕ defined as

i) **Tail of x.** The tail of x consists of the edges

$$t_3t_2, \quad t_3t_1, \quad t_2t_1, \quad t_2x, \quad t_1x, \quad t_1\text{TRUE}, \quad t_1\text{FALSE}.$$

ii) **Tails of the clause vertices.** For every clause C_j, introduce the edges

$$C_j^3C_j^2, \quad C_j^3C_j^1, \quad C_j^2C_j^1, \quad C_j^2C_j, \quad C_j^1C_j, \quad C_j^1L_{j_1}, \quad C_j^1L_{j_2}, \quad C_j^1L_{j_3},$$

where $C_j = \{\ell_{j_1}, \ell_{j_2}, \ell_{j_3}\}$.

iii) **Edges incident to clause vertices.** For every clause C_j and every literal $\ell_i \in C_j$, introduce the edge C_jL_i. Also introduce C_jTRUE and C_jFALSE. Finally, for every literal ℓ_k that is not the complement of a literal in C_j, introduce the edge C_jA_{ik}.

iv) **Edges among the remaining vertices.** All vertices in

$$\{L_i : 1 \le i \le 2n\} \cup \{A_{ik} : \ell_i \neq \overline{\ell_k}\} \cup \{\text{TRUE}, \text{FALSE}, x\}$$

are adjacent to one another, except for pairs of complementary literals. Thus, $L_iL_k \in E_\phi$ whenever ℓ_i and ℓ_k are not complementary, while $L_iL_k \notin E_\phi$ whenever ℓ_i and ℓ_k are complementary.

□

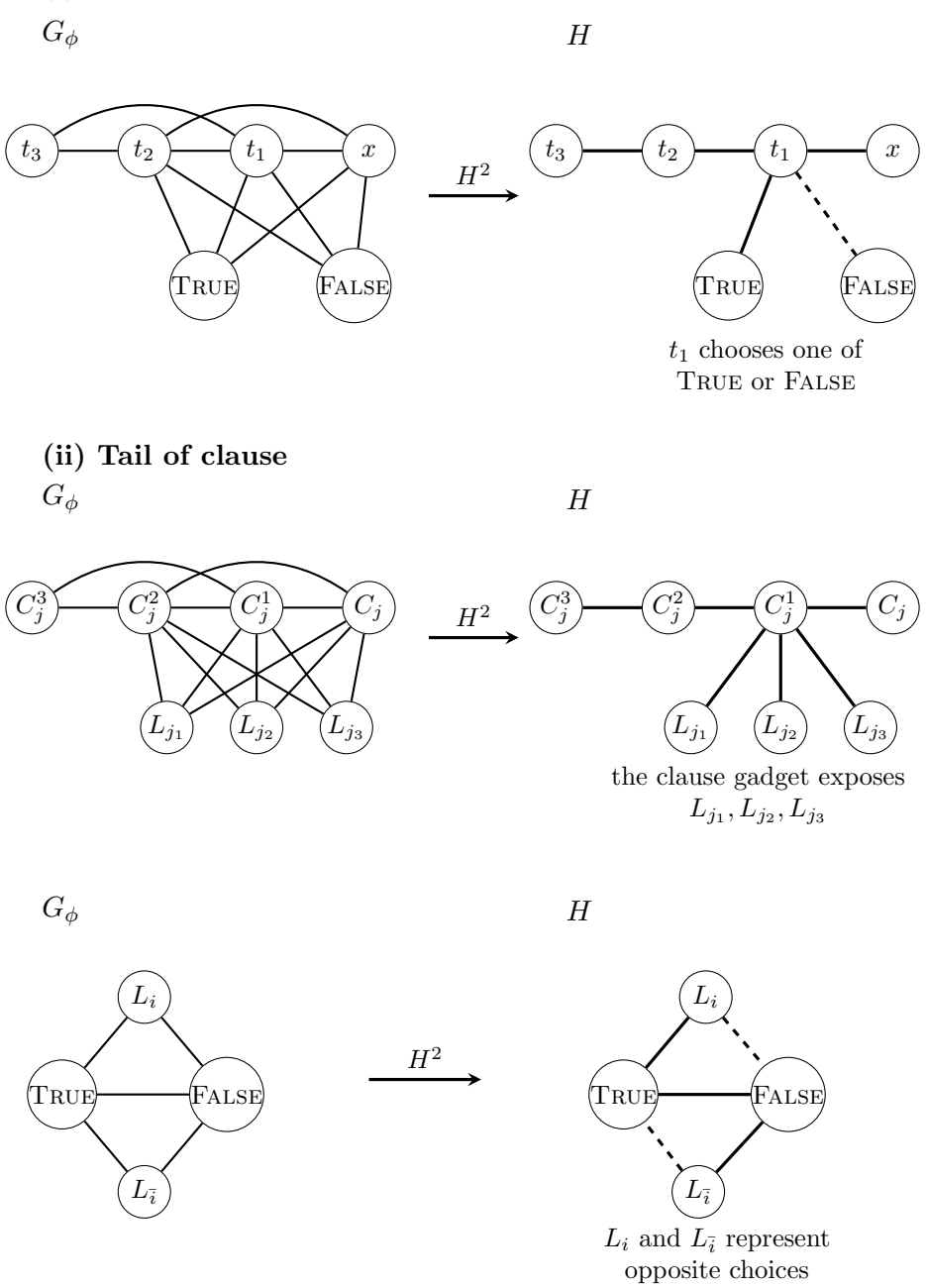


Figure 2.1: The gadgets illustrate how the structure of G_ϕ constrains a square root H. The tail of x forces a root path and leaves a TRUE/FALSE choice. The clause tail similarly forces a root path and exposes the three literal vertices of the clause. Complementary literal vertices are constrained to represent opposite truth values and mirror the example in Figure 1.2 where $(0, 1, 2, 3) = (L_i, L_{\bar{i}}, \text{FALSE}, \text{TRUE})$. Thus, the local structure of G_ϕ encodes the truth assignment represented by a square root.

The construction is polynomial in $n + m$ and depends only on the formula ϕ. Therefore it can be carried out without knowing whether ϕ has a not-all-equal satisfying assignment. Moreover, the number of vertices and edges introduced is polynomial in the size of ϕ.

The following lemma is the central combinatorial tool for proving the correctness of the reduction.

Lemma 1 (Tail Lemma)**.** *Let G be a connected graph containing four distinct vertices a, b, c, d such that*

$$N_G(a) = \{b, c\}, \quad N_G(b) = \{a, c, d\}, \quad cd \in E_G,$$

and suppose c has a neighbour in G outside $\{a, b, c, d\}$. If $G = H^2$, then

$$N_H(d) \setminus \{a, b, c, d\} = N_G(c) \setminus \{a, b, c, d\}.$$

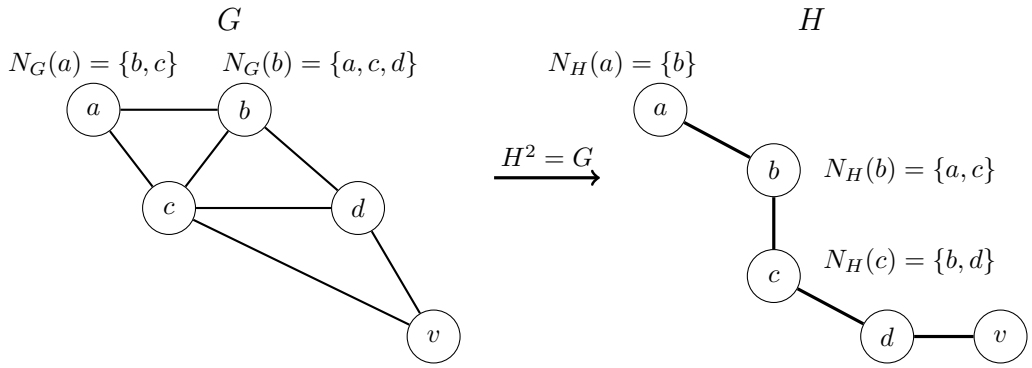


Figure 2.2: The forced local structure in the Tail Lemma. The square root H contains the path $a - b - c - d$. Consequently, $ac, bd \in E_G$ as distance-two edges. For an external neighbour v of c, the Tail Lemma gives $dv \in E_H$, and hence $dv \in E_G$.

Proof. Since $N_G(a) = \{b, c\}$, every neighbour of a in H is contained in $\{b, c\}$.

First show that $N_H(a) = \{b\}$. Suppose, for contradiction, that $N_H(a) = \{c\}$. Then $ac \in E_H$. Because $ab \in E_G = E_{H^2}$, there is a path of length at most two from a to b in H. As a only has the neighbour c, this forces $bc \in E_H$.

Now $cd \in E_G$, so c and d are at distance at most two in H. As $bc \in E_H$, the only possible P_3 from c to d uses b, and hence $bd \in E_H$.

Let $v \notin \{a, b, c, d\}$ be the neighbour of c in G. Since $cv \in E_G$, there is a path of length at most two from c to v in H. This path cannot go through a, otherwise $av \in E_G$, contradicting $N_G(a) = \{b, c\}$. Nor can it go through b, since that would give $bv \in E_G$, contradicting $N_G(b) = \{a, c, d\}$. Therefore the path must be $c - d - v$, giving $dv \in E_H$.

Since $bd, dv \in E_H$, have a path $b - d - v$ of length two in H, which implies $bv \in E_G$. This again contradicts $N_G(b) = \{a, c, d\}$. The contradiction eliminates the possibility $N_H(a) = \{c\}$, and consequently

$$N_H(a) = \{b\}. \tag{2.1}$$

Now we determine the neighbourhood of c. Since $ab \in E_H$ as $N_H(a) = \{b\}$, and $ac \in E_G$, the edge ac in $G = H^2$ must be realized by the path $a - b - c$. This gives that $bc \in E_H$. If b had a neighbour $x \notin \{a, c\}$ in H, then $a - b - x$ would be a P_3, forcing $ax \in E_G$, which contradicts $N_H(a) = \{b\}$. Thus

$$N_H(b) = \{a, c\}. \tag{2.2}$$

Since $bd \in E_G$ and $N_H(b) = \{a, c\}$, the P_3 from b to d in H must be $b - c - d$ and it cannot use a, as $N_H(a) = \{b\}$. Therefore $cd \in E_H$.

If c had a neighbour $y \notin \{b, d\}$ in H, then $b - c - y$ would give $by \in E_G$, contradicting Equation (2.2). Hence

$$N_H(c) = \{b, d\}. \tag{2.3}$$

It remains to relate the external neighbours of c in G to those of d in H. Take any $v \notin \{a, b, c, d\}$. If $cv \in E_G$, then there is a path of length at most two from c to v in H. By Equation (2.3), such a path must use b or d as the intermediate vertex. It cannot use b, since that would imply $bv \in E_G$, contradicting $N_G(b) = \{a, c, d\}$. Hence the path is $c - d - v$, so $dv \in E_H$. Conversely, if $dv \in E_H$, then $c - d - v$ is a P_3 in H since $cd \in E_H$, giving $cv \in E_G$.

Therefore have established that for every $v \notin \{a, b, c, d\}$,

$$v \in N_G(c) \text{ if and only if } v \in N_H(d).$$

The desired set equality follows immediately. □

We now establish the two directions required for the validity of the reduction.

Lemma 2. *If ϕ is a **Yes**-instance of* NAESAT*, then G_ϕ has a graph root.*

Proof Sketch. Suppose that ϕ has an assignment satisfying not-all-equal. We construct a graph root $H = (V_\phi, E_H)$ on the same vertex set as G_ϕ as follows.

The edges of H consist of the edges of the tails, together with edges connecting x to TRUE and FALSE. For each clause vertex C_j, connect C_j to the three literal vertices corresponding to the literals contained in the clause, together with the edges required by its tail. Each literal vertex is connected to TRUE if its corresponding literal is assigned TRUE and to FALSE if its corresponding literal is assigned FALSE. Finally, each literal vertex is connected to the appropriate literal-pair vertices, and each literal-pair vertex is connected to both TRUE and FALSE.

Since the assignment is not-all-equal, every clause contains at least one TRUE literal and at least one FALSE literal. Thus, the construction produces the required adjacencies between the clause vertices and their literals. Moreover, complementary literals do not have a common neighbour in H, while every pair of non-complementary literals has a common neighbour through its corresponding literal-pair vertex. Hence, the literal adjacencies in H^2 are exactly those specified in G_ϕ. The tail construction similarly ensures that all required tail adjacencies are obtained in H^2, while the connections to TRUE and FALSE produce the remaining required adjacencies.

Hence, $G_\phi = H^2$, and therefore G_ϕ has a graph root. □

Lemma 3. *If G_ϕ has a graph root, then ϕ is a **Yes**-instance of* NAESAT

Proof Sketch. Suppose that G_ϕ has a graph root H, so that $G_\phi = H^2$.

Apply Lemma 1 to the tail of x, with $(a, b, c, d) = (t_3, t_2, t_1, x)$. Since t_1 has the external neighbours TRUE and FALSE in G_ϕ, the lemma gives

$$N_H(x) \setminus \{t_3, t_2, t_1, x\} = \{\text{TRUE}, \text{FALSE}\}.$$

Thus, apart from its own tail, x is adjacent in H precisely to TRUE and FALSE.

Similarly, for each clause C_j, apply Lemma 1 to its tail with $(a, b, c, d) = (C_j^3, C_j^2, C_j^1, C_j)$. Since C_j^1 has the external neighbours $L_{j_1}, L_{j_2}, L_{j_3}$ in G_ϕ, the lemma gives

$$N_H(C_j) \setminus \{C_j^3, C_j^2, C_j^1, C_j\} = \{L_{j_1}, L_{j_2}, L_{j_3}\}.$$

Hence, apart from its own tail, C_j is adjacent in H precisely to the three literal vertices occurring in C_j. Since every literal vertex L_i is adjacent to x in $G_\phi = H^2$, each L_i must be adjacent in H to at least one of TRUE and FALSE.

On the other hand, complementary literal vertices are non-adjacent in G_ϕ. Therefore they cannot have a common neighbour in H, and hence a literal and its complement cannot both be adjacent to the same one of TRUE and FALSE. It follows that every literal vertex is adjacent to exactly one of TRUE and FALSE, and complementary literals are adjacent to opposite constant vertices.

We may therefore define an assignment by declaring a literal ℓ_i to be TRUE only when $L_i\text{TRUE} \in E_H$. Consider any clause $C_j = \{\ell_{j_1}, \ell_{j_2}, \ell_{j_3}\}$. By construction, C_j is adjacent in G_ϕ to both TRUE and FALSE. Since, apart from its own tail, C_j is adjacent in H only to $L_{j_1}, L_{j_2}, L_{j_3}$, these three literal vertices must include one adjacent to TRUE and one adjacent to FALSE.

Thus every clause contains at least one TRUE literal and at least one FALSE literal. Therefore the assignment is a not-all-equal satisfying assignment of ϕ, and hence

$$\phi \in \text{NAESAT}.$$

□

Theorem 2 (**NP**-complete)**.** *Problem* (2.1.1) *is* **NP***-complete.*

Proof. Theorem (1) shows that Problem (2.1.1) belongs to **NP**. For **NP**-hardness, consider the polynomial reduction from NAESAT to **SQUARE** given in Proof (2.1.1). By Lemma (2), every **Yes**-instance ϕ of NAESAT produces a graph G_ϕ with a graph root. Together with Lemma (3), this gives that NAESAT $\leq_p$ SQUARE, so Problem (2.1.1) is **NP**-hard.

Since Problem (2.1.1) belongs to **NP** and is **NP**-hard, it is **NP**-complete. □

2.2 Algorithm for Tree Roots

Ross and Harary give a necessary and sufficient characterization of graphs that are squares of trees, together with an algorithm for constructing the corresponding tree square root [RH60]. The characterization is based on the maximal cliques of the square and the way in which these cliques intersect. We first establish the structural properties of the square of a tree that underlie the characterization, before describing their root finding algorithm.

Theorem 3 (Structural Characterization of Squares of Trees)**.** *Let G be a finite, simple, undirected graph. Then $G = T^2$ for some tree T if and only if either G is a complete graph, or the maximal cliques $C_1, \ldots, C_n$ of G satisfy the following conditions:*

(i) G is connected and the closed neighbourhood of every vertex of T forms a clique in G.

(ii) If two maximal cliques have exactly one common vertex v, then there is another maximal clique containing v and one additional vertex from each of the two cliques.

(iii) Each maximal clique C_i can be associated with a unique multicliqual vertex b_i. The number of multicliqual vertices contained in C_i is equal to the number of maximal cliques containing b_i.

(iv) For any two maximal cliques C_i and C_j, $|C_i \cap C_j| \leq 2$, that is their intersection contains at most two points

(v) The number of pairs of maximal cliques whose intersection contains two vertices is $n - 1$.

To establish this characterization, let G be the square of a tree T and first consider several properties.

Lemma 4. *G is complete if and only if T is a star*

Proof. Suppose that T has a vertex v that is adjacent to every other vertex. Then every pair of vertices in T is at distance at most two, since any two vertices are either adjacent or have v as a common neighbour. Hence $G = T^2$ is complete.

Conversely, suppose that $G = T^2$ is complete. Then every pair of vertices in T has distance at most two, so $\text{diam}(T) \leq 2$. Since T is a tree, this implies that there is a vertex v adjacent to every other vertex. Thus, choosing v as the root gives a rooted tree T of depth one. □

Lemma 5. *G is connected and the closed neighbourhood of every vertex of T forms a clique in G.*

Proof. Let $v \in V_T$. Every pair of vertices in the closed neighbourhood $N_T[v]$ are either adjacent in T or have v as a common neighbour and thus has distance at most two in T. Hence $N_T[v]$ forms a clique in $G = T^2$. Furthermore, every edge of T is also an edge of G, so T is a spanning subgraph of G. Since T is connected, this implies G is also connected. □

Lemma 6. *If G is not complete, then the maximal cliques of G are precisely the closed neighbourhoods of the non-leaf vertices of T.*

Proof. Let v be a non-leaf vertex of T. Since v has at least two neighbours, every pair of vertices in $N_T(v)$ is at distance two through v. Hence $C(v) = N_T[v]$ is a clique in G. Now consider a vertex $u \notin N_T[v]$. The unique path from u to v in T has length at least two. If this path has length greater than two, then u is not adjacent in G to v. If the path has length two, then u can only be adjacent in G to the vertices along the corresponding part of the path. In either case, u cannot be added to $N_T[v]$ whilst maintaining a clique. Therefore $N_T[v]$ is a maximal clique.

Conversely, let C be a maximal clique of G. Since $G = T^2$, every pair of vertices in C are at distance at most two in T. The vertices of such a clique must therefore be contained in the closed neighbourhood of a common non-leaf vertex v of T. Thus $C \subseteq N_T[v]$. Since $N_T[v]$ is itself a clique and C is maximal, this gives $C = N_T[v]$. Hence every maximal clique of G is the closed neighbourhood of a non-leaf vertex of T. □

Lemma 7. *If G is not complete, then a vertex $v \in V_G$ belongs to exactly one maximal clique if and only if v is a leaf of T.*

Proof. Let v be a leaf of T, and let u be its unique neighbour. By Lemma 6, every maximal clique corresponds to the closed neighbourhood of a non-leaf vertex. Since v has only one neighbour, the only maximal clique containing v is $N_T[u]$.Thus v belongs to exactly one maximal clique.

Conversely, suppose that v is not a leaf. Then v has at least two neighbours in T. By Lemma 6, $N_T[v]$ is a maximal clique containing v. Since G is not complete, Lemma 4 implies that T is not a star. Hence at least one neighbour u of v is a non-leaf vertex. Therefore the closed neighbourhood $N_T[u]$ is another maximal clique containing v. Thus v belongs to at least two maximal cliques and the vertices belonging to exactly one maximal clique are exactly the leaves of T. □

Lemma 8. *Let C_i and C_j be maximal cliques corresponding to non-leaf vertices v_i and v_j of T. Then $|C_i \cap C_j| = 2$ if and only if $v_i v_j \in E_T$.*

Proof. Suppose that v_i and v_j are adjacent in T. Then both v_i and v_j belong to $N_T[v_i] \cap N_T[v_j]$. If there were another vertex u in this intersection, then u would be adjacent to both v_i and v_j in T. Together with the edge v_iv_j, this would create a cycle in T, which is impossible. Therefore, $|N_T[v_i] \cap N_T[v_j]| = 2$. By Lemma 6, these closed neighbourhoods are the corresponding maximal cliques, so $|C_i \cap C_j| = 2$.

Conversely, suppose that $|C_i \cap C_j| = 2$. Since v_i and v_j belong to their respective closed neighbourhoods, the intersection implies that v_i and v_j are either adjacent or have a common neighbour in T. If they are not adjacent, there would be a common neighbour u adjacent to both v_i and v_j. Then $v_i, v_j, u \in C_i \cap C_j$, which would give an intersection of at least three vertices, contradicting $|C_i \cap C_j| = 2$. Therefore, v_i and v_j must be adjacent with $v_iv_j \in E_T$. □

Combining gives the following proof of Theorem (3)

Proof. Suppose first that $G = T^2$ for some tree T. If G is complete, then Lemma 4 shows that T is a star.

Now suppose that G is not complete. By Lemma 5, G is connected and every closed neighbourhood in T forms a clique in G, giving (i).

By Lemma 6, each maximal clique of G is the closed neighbourhood of a non-leaf vertex of T. Suppose that two maximal cliques $N_T[u]$ and $N_T[v]$ have exactly one common vertex x. Since T is a tree, u and v cannot be adjacent, and hence x lies on the unique path between u and v. Thus, x is adjacent in T to both u and v, so $N_T[x]$ forms a maximal clique containing x, together with the additional vertices u and v from the two original cliques. This gives (ii).

For a maximal clique $C_i = N_T[b_i]$, the associated vertex b_i is the corresponding non-leaf vertex of T. Every non-leaf neighbour of b_i in T gives another maximal clique containing b_i, while each such neighbour is itself also a multicliqual vertex contained in C_i. Consequently, the number of multicliqual vertices contained in C_i is equal to the number of maximal cliques containing b_i, giving (iii).

By Lemma 8, two maximal cliques intersect in two vertices exactly when their corresponding non-leaf vertices are adjacent in T. Since T contains no cycles, two such closed neighbourhoods cannot have more than two vertices in common. This gives (iv). Finally, the non-leaf vertices of T, together with the edges between them, form a tree. If there are n such vertices, there are exactly $n - 1$ edges between them. By Lemma 8, each such edge corresponds to a pair of maximal cliques with an intersection containing exactly two vertices. Hence (v) holds.

Conversely, suppose that G is not complete and that its maximal cliques $C_1, \ldots, C_n$ satisfy (i)–(v). By (iii), we may associate each maximal clique C_i with a unique vertex b_i that belongs to multiple maximal cliques. These vertices will form the internal vertices of the tree root. Construct T with vertex set $V_T = \{b_1, \ldots, b_n\}$. For every pair of maximal cliques satisfying $|C_i \cap C_j| = 2$, add the edge b_ib_j to T. By (v), T contains exactly $n - 1$ edges, and the intersection structure given by the characterization ensures that these edges connect all vertices without forming a cycle. Thus T is a tree. It remains to add the unicliqual vertices. By (iii), these vertices correspond to leaves of the root. For each clique C_i, attach all vertices of C_i that occur in no other maximal clique as leaves adjacent to b_i. The resulting graph is a tree T. Each maximal clique C_i is represented by the closed neighbourhood of b_i in T, while the remaining vertices are attached as leaves to their corresponding internal vertices. Therefore every edge of G is represented by two vertices at distance at most two in T, and no additional edges are introduced. Hence $T^2 = G$ and G is the square of a tree. □

The proof of Theorem (3) gives the following algorithm. Given a graph known to be the square of a tree, identify all maximal cliques, classify the vertices as multicliqual or unicliqual according

to the number of maximal cliques in which they occur, and construct the tree T by connecting the multicliqual vertices to form the internal structure of T and attaching the unicliqual vertices as leaves to their corresponding multicliqual vertices.

Algorithm 1 Ross-Harary Tree Construction

Input: A graph G where $G = T^2$
Output: A tree T

Step 1
if $G = K_p$ is complete **then**
    Return tree T with depth 1 where root can be any vertex
else
    Continue
end if

Step 2
Identify all maximal cliques $C_1, \ldots, C_n$ of G using an algorithm such as Bron-Kerbosch [BK73] where $n > 1$ from Theorem 3 (i)

Step 3
Let $b_1, \ldots, b_n$ be multicliqual points corresponding to $C_1, \ldots, C_n$ formed from Theorem 3 (iii)
Construct a graph T with vertex set $\{b_1, \ldots, b_n\}$
for each pair (C_i, C_j) satisfying $|C_i \cap C_j| = 2$ **do**
    Add edge between b_i and b_j in T
end for
By Theorem 3 (v), T is a tree

Step 4
for each clique C_i **do**
    Let n_i be the number of unicliqual points in C_i
    Attach n_i leaf nodes to b_i in T
end for
The resulting tree T has the same number of nodes as G and satisfies $T^2 = G$

Theorem 4 (Tree Root Algorithm)**.** *Given a graph $G = T^2$, Algorithm* (1) *returns a tree T such that $T^2 = G$. Furthermore, this tree is unique up to isomorphism.*

Proof. If G is complete, Step 1 returns a star. By Lemma 4, this is a tree square root of G. Otherwise, Lemma 6 shows that each maximal clique corresponds to a non-leaf vertex of the root. Thus the vertices $b_1, \ldots, b_n$ identified in Step 3 determine the internal vertices of T. By Lemma 8, two of these internal vertices are adjacent precisely when their corresponding maximal cliques have an intersection of size two. Therefore Step 3 recovers all edges between the internal vertices. The remaining vertices belong to exactly one maximal clique and, by Lemma 7, correspond to leaves. Step 4 therefore places each of these vertices at its unique position in the tree. Hence the algorithm reconstructs a tree T satisfying $T^2 = G$. Since the internal vertices, their adjacencies, and the attachment of every leaf are all determined by the maximal cliques of G, the resulting tree is unique up to isomorphism. □

While Algorithm 1 formalizes the original 1960 constructive proof by Ross and Harary, it predates the formalization of computational complexity classes $\mathcal{P}$ vs. $\mathcal{NP}$ established in the early

1970s by Cook [Coo71] and Karp [Kar72]. From a modern perspective, Ross and Harary's strategy relies on global maximal clique enumeration to group vertices into local star subgraphs. If implemented using general-purpose clique algorithms such as Bron-Kerbosch, this approach suffers from exponential worst-case execution time.

In contrast, Lin and Skiena bypass clique enumeration entirely by shifting to a local, bottom-up leaf-trimming strategy to give a polynomial algorithm [LS95]. Rather than searching for global subgraphs, they identify leaves of T directly as simplicial vertices with minimal degree, partition their closed neighborhoods into leaf and non-leaf sets, and use the structural properties of these local neighborhoods to reconstruct parent-child adjacencies directly. By repeatedly trimming identified leaves, they reduce the problem to smaller tree squares until the tree structure is fully recovered.

2.3 Mathematical Optimization Approach

The two methods so far have primarily focused on the definition of a square of a graph to be paths of length at most three defined in Definition (1.3.39). Using adjacency matrices, this definition is equivalent expressed as

$$A_G = (A_H)^2 \vee A_H. \tag{2.4}$$

where $\vee$ is applied entrywise to the matrices. This concise interpretation with the logical operator lends itself easily to a SAT problem whereby one could introduce Boolean variables for the edges of H and encode $(A_H)^2_{ij} = \big((A_H)_{i1} \wedge (A_H)_{1j}\big) \vee \big((A_H)_{i2} \wedge (A_H)_{2j}\big) \vee \cdots \vee \big((A_H)_{in} \wedge (A_H)_{nj}\big)$. Thus, for each pair of distinct vertices i, j, the condition that ij is an edge of G can be expressed as

$$g_{ij} \text{ if and only if } \big(\bigwedge_k (x_{ik} \wedge x_{kj}) \vee x_{ij}\big). \tag{2.5}$$

The resulting Boolean formula can be converted into a standard SAT instance using auxiliary variables and a polynomial number of clauses. Hence, the logical formulation provides a polynomial-size SAT representation of the graph-square condition.

However, constructing a polynomial-size SAT instance does not imply that the underlying problem can be solved in polynomial time. A general SAT instance may require exponential time to solve, since SAT is **NP**-complete. Therefore, to obtain a polynomial-time algorithm through this approach, one would additionally need to show that the SAT instances arising from Equation (2.5) belongs to a tractable subclass of SAT, such as 2-SAT, or otherwise provide a polynomial-time algorithm for solving these particular instances. The formulation in Equation (2.5) evidently does not belong to 2-SAT, as each constraint involves a disjunction over all possible common neighbours k and therefore contains terms involving more than two Boolean variables. More generally, one would suspect from the results in Section 2.1 that there is no immediate polynomial-time algorithm for solving this instance and any restricted version of the graph root finding problem would necessitate significant machinery.

Instead, we aim to situate the problem completely within a polynomial-time framework by considering another equivalent representation of Equation (2.4) and its equivalent formulation in Definition (1.3.39).

Lemma 9. *Let G and H be graphs with adjacency matrices A_G and A_H,respectively. The graph H is a root of G if and only if Equation 1.2 holds*

$$A_G = \mathbb{1}_{\geq 1}\big((A_H)^2 - D_H + A_H\big).$$

Proof. For $i \neq j$, since D_H is diagonal, have $(D_H)_{ij} = 0$ and hence

$$\big((A_H)^2 - D_H + A_H\big)_{ij} = (A_H)^2_{ij} + (A_H)_{ij}.$$

Suppose $ij \in E_G$ such that $(A_H)^2_{ij} + (A_H)_{ij} \geq 1$. This implies that at least one of $(A_H)_{ij} = 1$ or $(A_H)^2_{ij} \geq 1$ must be true. If $(A_H)_{ij} = 1$, then $d_H(i,j) = 1$. If $(A_H)^2_{ij} \geq 1$, then there exists a vertex k such that $ik, kj \in E_H$, and hence $d_H(i,j) \leq 2$. Therefore $1 \leq d_H(i,j) \leq 2$, and thus $ij \in E_{H^2}$.

Suppose $ij \in E_{H^2}$ such that $1 \leq d_H(i,j) \leq 2$. D_H is diagonal such that $(D_H)_{ij} = 0$. If $d_H(i,j) = 1$, then $(A_H)_{ij} = 1$. If $d_H(i,j) = 2$, then there exists a vertex k such that $ik, kj \in E_H$, and hence $(A_H)^2_{ij} \geq 1$. In either case, $(A_H)^2_{ij} + (A_H)_{ij} \geq 1$, so $(A_G)_{ij} = 1$, and hence $ij \in E_G$.

For $i = j$, since all graphs are simple, have $(E.)_{ii} = 0$, $(A.)_{ii} = 0$ and hence

$$\big((A_H)^2 - D_H + A_H\big)_{ii} = (A_H)^2_{ii} - (D_H)_{ii}.$$

$(A_H)^2_{ii} - (D_H)_{ii} = 0$ is always true since the degree matrix of H always has diagonal values $(D_H)_{ii} = \sum_k x_{ik}$ and each edge from vertex i to its neighbour in H will always create a loop to itself in the square and so $(A_H)^2_{ii} = \sum_{k=1} x_{ik}x_{ki} = \sum_{k=1} x^2_{ik}$ by the symmetry of adjacency matrix A_H.

Since $x_{ik} \in \{0,1\}$, the square of the boolean is identical to the boolean $x_i k$ such that $(A_H)^2_{ii} = \sum_k x_{ik} = (D_H)_{ii}$ and $(A_G)_{ii} = \mathbb{1}_{\geq 1}\big((A_H)^2 - D_H + A_H\big)$ as required.

Combining both gives that $E_G = E_{H^2}$, and so $G = H^2$. □

The formulation of the adjacency matrix of the graph square in Equation (1.2) is very powerful. However, the presence of the indicator function introduces a nonlinearity that prevents the problem from being expressed as a purely quadratic form, thereby limiting the direct applicability of existing results from the quadratic optimization literature.

Lemma 10. *If H has girth 5, then Equation (1.2) can be simplified to become*

$$A_G = \big((A_H)^2 - D_H + A_H\big). \tag{2.6}$$

Proof. Since the graph root has girth 5 there are no cycles of length 3 or 4 in H. Thus for every pair of vertices $i \neq j \in V$ there is either no cycle that contains both i and j or i and j lie on a common cycle of length at least 5. In all cases $(D_H)_{ij} = 0$. If there is no cycle, then there is at most one path between i and j.

i) If the path is of length 2, then $(A_H)_{ij} = 1, (A_H)^2_{ij} = 0$ and so $(A_G)_{ij} = 0 - 0 + 1 = 1$.

ii) If the path is of length 3, then $(A_H)_{ij} = 0, (A_H)^2_{ij} = 1$ and so $(A_G)_{ij} = 1 - 0 - 0 = 1$.

iii) If the path is of length ≥ 3 or a path does not exist, then $(A_H)_{ij} = 0, (A_H)^2_{ij} = 0$ which implies that $(A_G)_{ij} = 0$.

If i and j lie on a common cycle of length at least five then there are two paths between i, j

i) If one path is of length 2, then the other is of length ≥ 4 and hence $(A_H)_{ij} = 1, (A_H)^2_{ij} = 0$ which implies that $(A_G)_{ij} = 0 - 0 + 1 = 1$.

ii) If one path is of length 3, then the other is of length ≥ 3 and hence $(A_H)_{ij} = 0, (A_H)^2_{ij} = 1$ which implies that $(A_G)_{ij} = 1 - 0 - 0 = 1$.

iii) If the paths are both of length ≥ 3, then $(A_H)_{ij} = 0, (A_H)^2_{ij} = 0$ which implies that $(A_G)_{ij} = 0$.

If $i = j$, then $(A_H)_{ii} = 0$, $(D_H)_{ii} = \deg_H(i)$, and $(A_H)^2_{ii} = \deg_H(i)$. Thus,

$$(A_H)^2_{ii} - (D_H)_{ii} + (A_H)_{ii} = \deg_H(i) - \deg_H(i) + 0 = 0 = (A_G)_{ii}.$$

□

Rather than focusing on the restricted class given by the integer program of Equation (2.6), we aim for a unified method for solving the problem in general. Nevertheless this result gives strong intuition into the results established by Farzad, Lau, Le, and Tuy in [Far+09] in constructing a polynomial time algorithm for recognizing graph roots where H has girth at least 6.

2.3.1 Formulation

Using Lemma (9), we construct an integer program to solve instances of **SQUARE**. The set of decision variables is

$$X = \{x_{ij} \mid i, j \in V_G\},$$

where $x_{ij} \in \{0, 1\}$ denotes the corresponding entry of the adjacency matrix A_H. In a feasible solution, $x_{ij} = 1$ if and only if $ij \in E_H$. The variables $y_{ikj} \in \{0, 1\}$ represent the products $x_{ik}x_{kj}$ in $(A_H)^2_{ij}$, while $z_{ij} \in \mathbb{Z}_{\geq 0}$ represents the argument of the indicator function.

Given an input G to **SQUARE**, our goal is to construct the following matrices, where g_{ij} is the corresponding entry of the adjacency matrix of G.

$$D_H = \begin{bmatrix} \deg_H(1) & 0 & \cdots & 0 & 0 \\ 0 & \ddots & \vdots & \ddots & \vdots \\ \vdots & \cdots & \deg_H(i) & \cdots & 0 \\ 0 & \ddots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & \cdots & \deg_H(n) \end{bmatrix}, A_H = \begin{bmatrix} x_{11} & \cdots & x_{1j} & \cdots & x_{1n} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{i1} & \cdots & x_{ij} & \cdots & x_{in} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{nj} & \cdots & x_{nn} \end{bmatrix} \text{ such that}$$

$$(A_H)^2 = \begin{bmatrix} \sum_{k=1}^{n} x_{1k}x_{k1} & \cdots & \sum_{k=1}^{n} x_{1k}x_{kj} & \cdots & \sum_{k=1}^{n} x_{1k}x_{kn} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \sum_{k=1}^{n} x_{ik}x_{k1} & \cdots & \sum_{k=1}^{n} x_{ik}x_{kj} & \cdots & \sum_{k=1}^{n} x_{ik}x_{kn} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \sum_{k=1}^{n} x_{nk}x_{k1} & \cdots & \sum_{k=1}^{n} x_{nk}x_{kj} & \cdots & \sum_{k=1}^{n} x_{nk}x_{kn} \end{bmatrix}.$$

Then with known constants $g_{ij} \in \{0, 1\}$ from A_G, we can formulate Equation (1.2) as an integer

program by equating entries of each matrix to give the following linear system of n^2 equations.

$$\begin{bmatrix} g_{11} \\ \vdots \\ g_{1j} \\ \vdots \\ g_{1n} \\ \vdots \\ g_{ij} \\ \vdots \\ g_{nn} \end{bmatrix} = \begin{bmatrix} \mathbb{1}_{\geq 1}(z_{11}) \\ \vdots \\ \mathbb{1}_{\geq 1}(z_{1j}) \\ \vdots \\ \mathbb{1}_{\geq 1}(z_{1n}) \\ \vdots \\ \mathbb{1}_{\geq 1}(z_{ij}) \\ \vdots \\ \mathbb{1}_{\geq 1}(z_{nn}) \end{bmatrix} = \begin{bmatrix} \mathbb{1}_{\geq 1}\big(\sum_{k=1}^{n} x_{1k}x_{k1} - (D_H)_{11} + x_{11}\big) \\ \vdots \\ \mathbb{1}_{\geq 1}\big(\sum_{k=1}^{n} x_{1k}x_{kj} - (D_H)_{1j} + x_{1j}\big) \\ \vdots \\ \mathbb{1}_{\geq 1}\big(\sum_{k=1}^{n} x_{1k}x_{kn} - (D_H)_{1n} + x_{1n}\big) \\ \vdots \\ \mathbb{1}_{\geq 1}\big(\sum_{k=1}^{n} x_{ik}x_{kj} - (D_H)_{ij} + x_{ij}\big) \\ \vdots \\ \mathbb{1}_{\geq 1}\big(\sum_{k=1}^{n} x_{nk}x_{kn} - (D_H)_{nn} + x_{nn}\big) \end{bmatrix}. \tag{2.7}$$

The matrix D_H is diagonal can be represented as $(D_H)_{ii} = \sum_k x_{ik}$ where $(D_H)_{ij} = 0$ for $i \neq j$. The variable z_{ij} represent the terms inside the indicator function such that

$$g_{ij} = 1 \text{ if and only if } z_{ij} \geq 1 \text{ and } g_{ij} = 0 \text{ if and only if } z_{ij} = 0.$$

Let $M \in \mathbb{Z}_{>0}$ be a sufficiently large constant that may be bounded by the number of edges, that is $M \leq n(n-1) = O(n^2)$. The above condition can be modelled using a standard big-M formulation. The resulting integer program is given by

Objective Function:

$$\max \quad 0 \tag{2.8}$$

Variables:

$$x_{ij} \in \mathbb{Z}_{\geq 0}, \qquad \forall i,j \in \{1,\dots,n\}, \tag{2.9}$$

$$y_{ikj} \in \mathbb{Z}_{\geq 0}, \qquad \forall i,k,j \in \{1,\dots,n\}, \tag{2.10}$$

$$z_{ij} \in \mathbb{Z}_{\geq 0}, \qquad \forall i,j \in \{1,\dots,n\}, \tag{2.11}$$

Constraints:

(1) Indicator argument:

$$z_{ii} = \sum_{k=1}^{n} y_{iki} - \sum_{k=1}^{n} x_{ik}, \forall i \in \{1,\dots,n\}, \tag{2.12}$$

$$z_{ij} = \sum_{k=1}^{n} y_{ikj} + x_{ij}, \qquad \forall i \neq j \in \{1,\dots,n\}, \tag{2.13}$$

(2) Indicator function:

$$z_{ij} \leq M g_{ij}, \qquad \forall i,j \in \{1,\dots,n\}, \tag{2.14}$$

$$z_{ij} \geq g_{ij}, \qquad \forall i,j \in \{1,\dots,n\}, \tag{2.15}$$

(3) Product:

$$y_{ikj} \leq x_{ik}, \qquad \forall i,k,j \in \{1,\dots,n\}, \tag{2.16}$$

$$y_{ikj} \leq x_{kj}, \qquad \forall i,k,j \in \{1,\dots,n\}, \tag{2.17}$$

$$y_{ikj} \geq x_{ik} + x_{kj} - 1, \qquad \forall i,k,j \in \{1,\dots,n\}, \tag{2.18}$$

(4) Upper Bound:

$$x_{ij} \leq 1, \qquad \forall i,j \in \{1,\dots,n\}, \tag{2.19}$$

$$y_{ikj} \leq 1, \qquad \forall i,k,j \in \{1,\dots,n\}, \tag{2.20}$$

(5) Symmetry:

$$x_{ij} = x_{ji}, \qquad \forall i,j \in \{1,\dots,n\}, \tag{2.21}$$

(6) Diagonal:

$$x_{ii} = 0, \qquad \forall i \in \{1,\dots,n\}. \tag{2.22}$$

The integer program on the arbitrary graph G using Equations (2.9)–(2.22) together with the constant objective function Equation (2.8) is denoted as $\mathcal{IP}(G)$.

The constraints in Equations (2.21)–(2.22) impose the required symmetry and diagonal structure on the adjacency matrix A_H. Although analogous constraints could also be imposed on the auxiliary variables z, they are omitted as they are redundant and may introduce artificial degeneracy into the linear program.

2.3.2 Validity of $\mathcal{IP}(G)$

Theorem 5. *Let G be a graph and consider the integer program whose variables $x_{ij} \in \{0,1\}$ encode a graph H with adjacency matrix $A_H = (x_{ij})$. Then G has a square root H if and only if the integer program has a feasible solution x. The feasible solution x defines entries of the adjacency matrix of a graph H that satisfies $H^2 = G$.*

Proof. Suppose G has a square root H and let A_H be the adjacency matrix of H and let $x_{ij} = (A_H)_{ij}$. Construct the variables x_{ij}, y_{ikj}, z_{ij} and the constant g_{ij} as follows.

i) Let $x_{ij} = 1$ if $ij \in E_H$ and $x_{ij} = 0$ otherwise.

ii) Let $y_{ikj} = x_{ik}x_{kj}$ for all i,k,j.

iii) Let $z_{ij} = \sum_{k=1}^{n} y_{ikj} + x_{ij}$ for all i,j.

iv) Let $z_{ii} = \sum_{k=1}^{n} y_{iki} - \sum_{k=1}^{n} x_{ik} = \sum_{k=1}^{n} x_{ik}^2 - \sum_{k=1}^{n} x_{ik} = 0$ since $x_{ij} \in \{0,1\} \implies x_{ik}^2 = x_{ik}$.

v) Let $g_{ij} = 1$ if $ij \in E_G$ and $g_{ij} = 0$ otherwise.

Verifying constraints of the integer program for the variables

i) As $x_{ij} \in \{0, 1\}$, then $y_{ikj} \in \{0, 1\}$, Equations (2.9)-(2.11) and Equations (2.19)-(2.20) are satisfied.

ii) As H is a simple directed graph, its adjacency matrix has zero diagonal and is symmetric. Therefore Equations (2.21) -(2.22) are satisfied.

iii) As $y_{ikj} = x_{ik}x_{kj}$ and all variables are binary. Therefore Equations (2.16)- (2.18) are satisfied.

iv) Equation (2.13) by definition is satisfied.

v) As $z_{ii} = \sum_{k=1}^{n} x_{ik}x_{ki} - \sum_{k=1}^{n} x_{ik}$, Equation (2.12) is satisfied.

vi) As $z_{ij} \geq 1$ if and only if $x_{ij} = 1$ or there exists k such that $x_{ik} = x_{kj} = 1$ and as $H^2 = G$, have $g_{ij} = 1$ if and only if $1 \leq d_H(i, j) \leq 2$. Thus $g_{ij} = 1$ if and only if $z_{ij} \geq 1$. Therefore indicator constraints Equations (2.14)-(2.15) hold.

All constraints of the integer program are satisfied by the constructed variables. Hence, the integer program is feasible.

Suppose the integer program on graph G has a feasible solution x, and let x_{ij} define a graph H by

$$ij \in E_H \text{ if and only if } x_{ij} = 1.$$

As x_{ij} satisfies the constraints of the integer program,

i) Equation (2.9) and Equation (2.19) gives $x_{ij} \in \{0, 1\}$ for all i, j,

ii) Equation (2.21) and Equation (2.22) give $x_{ij} = x_{ji}$ and $x_{ii} = 0$,

such that x_{ij} defines a zero diagonal, symmetric matrix with all elements being either 0 or 1 which is a valid adjacency matrix of a graph. Examining the auxiliary variables y_{ikj} and z_{ij},

i) Equations (2.16)-(2.18) give the strict equality $y_{ikj} = x_{ik}x_{kj}$ for all i, j, k,

ii) Equation (2.13) gives $z_{ij} = \sum_{k=1}^{n} y_{ikj} + x_{ij} = \sum_{k=1}^{n} x_{ik}x_{kj} + x_{ij}$ for all $i \neq j$,

iii) Equation (2.12) gives $z_{ii} = \sum_{k=1}^{n} y_{iki} + \sum_{n}^{k=1} x_{ik} = \sum_{k=1}^{n} x_{ik}x_{ki} - \sum_{n}^{k=1} x_{ik} = 0$ for all i since

$$\sum_{k=1}^{n} x_{ik}x_{ki} = \sum_{k=1}^{n} x_{ik}^2 = \sum_{k=1}^{n} x_{ik},$$

by symmetry and since $x_{ij} \in \{0, 1\}$.

The indicator constraints Equations (2.14)-(2.15) thus give for $i \neq j$

$$g_{ij} = 1 \text{ if and only if } \sum_{k=1}^{n} x_{ik}x_{kj} + x_{ij} \geq 1 \text{ and } g_{ij} = 0 \text{ if and only if } \sum_{k=1}^{n} x_{ik}x_{kj} + x_{ij} = 0.$$

Since $x_{ij} \in \{0, 1\}$, this is equivalent to saying that $g_{ij} = 1$ if and only if $\Big(x_{ij} = 1\Big) \vee \Big($ there exists k such that $x_{ik} = 1 \wedge x_{kj} = 1\Big)$ and $g_{ij} = 0$ if and only if $x_{ij} = 0$. By definition of H this means that $g_{ij} = 1$ if and only if $\Big(ij \in E_H\Big) \vee \Big($ there exists k such that $ik, kj \in E_H\Big)$ which is equivalent to

$$g_{ij} = 1 \text{ if and only if } 1 \leq d_H(i, j) \leq 2.$$

Applying Lemma 9 thus gives $ij \in E_G$ if and only if $ij \in E_H^2$. Therefore $G = H^2$. □

In addition to the verification above, we verify the correctness of the integer program experimentally for all graphs on $n \leq 8$ vertices. The outcome of the computational verification is given in Table (2.1). The total number of simple graphs on $n = 2, 3, 4, \ldots$ unlabeled nodes is the sequence $2, 4, 11, 34, 156, 1044, 12346, \ldots$ given by `A000088`. The number of graphs which are squares is the sequence $2, 3, 6, 11, 28, 77, 307, \ldots$ given by `A382181`.

n	H exists	H does not exist
3	3	1
4	6	5
5	11	23
6	28	128
7	77	967
8	307	12039

Table 2.1: Outcomes of the integer program for graphs of different orders of n.

Chapter 3

Square of a Tree using Linear Programming

In this chapter, we explore the recognition of squares of trees using linear programming. We first introduce a linear programming relaxation for $\mathcal{IP}(G)$ and investigate the structure of its feasible solutions. We then consider additional constraints that can be derived from the structure of the input graph and be used to augment the relaxation. The resulting formulations are tested computationally on a collection of graphs in order to evaluate their ability to recover integral solutions corresponding to tree roots.

The methodology used in this work is implemented using the SageMath [The26] framework with Gurobi [Gur26] as the optimization backend. The barrier method is used, as seen in the code block Listings 2–10, through the parameter `Method=2`. The barrier method is an interior-point method that uses a predictor-corrector scheme based on Newton iterations to approach an optimal solution from within the feasible region. For linear programs, classical interior-point theory provides polynomial worst-case bounds on the number of iterations, with each iteration requiring polynomial computational effort in the number of variables and constraints [Gon91]. Thus, the barrier method provides a theoretically polynomial approach for solving the LP formulations considered in this chapter, although these worst-case bounds do not necessarily reflect the practical running time of the implementation.

Throughout this thesis, we have only considered unlabelled graphs, so graphs that are isomorphic are regarded as the same graph instance. However, the algorithms themselves operate on a labelled representation of each graph. Each unlabelled graph is therefore represented computationally by a particular labelled graph generated by SageMath, with the assigned vertex labels retained throughout the subsequent computations.

The particular labelling depends on the graph generation procedure. Consequently, different labellings of the same unlabelled graph, or different generated representatives of the same isomorphism class, may lead to different intermediate or selected solutions of the optimization procedures, even though the underlying graph structure is unchanged. The results reported in this chapter therefore correspond to the particular labelled representatives produced by the stated generation procedure. The complete test harness and instance generation scripts are available at `github.com/Hellwoodoo/Graph-Squares` and the functions used by the harness are reproduced in the appendix (Listings 1–10) for completeness.

3.1 Linear Program Relaxation

We construct the linear relaxation of the integer program defined by Equations (2.8)–(2.22), which generates a feasible weighted solution in polynomial time. The relaxation has identical constraints and the same constant objective function, however the decision variables Equations (2.9)–(2.11) are replaced with

$$x_{ij} \in \mathbb{R}_{\geq 0}, \qquad \forall i, j \in \{1, \ldots, n\}, \tag{3.1}$$
$$y_{ikj} \in \mathbb{R}_{\geq 0}, \qquad \forall i, k, j \in \{1, \ldots, n\}, \tag{3.2}$$
$$z_{ij} \in \mathbb{R}_{\geq 0}, \qquad \forall i, j \in \{1, \ldots, n\}. \tag{3.3}$$

This linear program on the arbitrary graph G using Equations (3.1)–(3.3) and Equations (2.12)–(2.22) is denoted as $\mathcal{LP}(G)$.

Such a linear program may have *integral solutions* and *fractional solutions* when the root exists and *fractional solutions* and *infeasible solutions* when the root does not exist.

Definition 3.1.1 (Integral Solutions)**.** *A feasible solution to the linear program is an* integral solution *if*

$$x_{ij} \in \{0, 1\} \quad \textit{for all } i, j \in V.$$

In this case, the variables x_{ij} define a graph root H through

$$E_H = \{(i, j) \in V \times V : x_{ij} = 1\}.$$

An integral solution to $\mathcal{LP}(G)$ is a feasible solution for $\mathcal{IP}(G)$.

Definition 3.1.2 (Fractional Solutions)**.** *A feasible solution to the linear program is a* fractional solution *if*

$$x_{ij} \in [0, 1] \quad \textit{for all } i, j \in V,$$

and there exists at least one pair (i, j) such that $0 < x_{ij} < 1$. In this case, the variables x_{ij} do not directly define a graph root.

Figure 3.1 gives the classification of solutions on all inputs of the graph of size $n = 8$. This can be condensed into Table 3.1 which gives the complete classification up to $n = 8$ vertices.

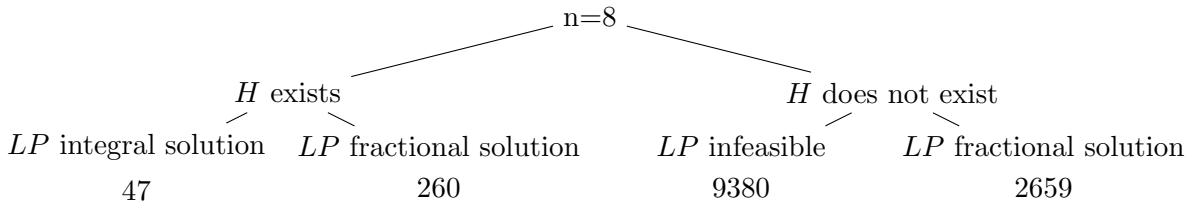


Figure 3.1: LP outcomes for $n = 8$ vertices using $\mathcal{LP}(G)$.

In Figure 1.3 solutions which are LP feasible may be integral or fractional. The results in Table 3.1 demonstrate both cases. When H exists, an integral LP solution corresponds to a feasible point in $\mathcal{F}_{IP}$, while a fractional LP solution lies in $\mathcal{F}_{LP} \setminus \mathcal{F}_{IP}$. For $n = 3$ and $n = 4$, all

[1]The LP integral solution sequence $2, 3, 6, 10, 17, 29, 51, \ldots$ matches A285665. However, it is unclear why this should be the case.

n	H exists		H does not exist	
	LP integral solution	LP fractional solution	LP infeasible	LP fractional solution
3	3	0	1	0
4	5	1	5	0
5	7	4	20	3
6	13	15	107	21
7	22	55	767	200
8	44	263	9380	2659

Table 3.1: Outcomes of the LP relaxation for graphs of different orders of n, classified according to whether a square root H exists.

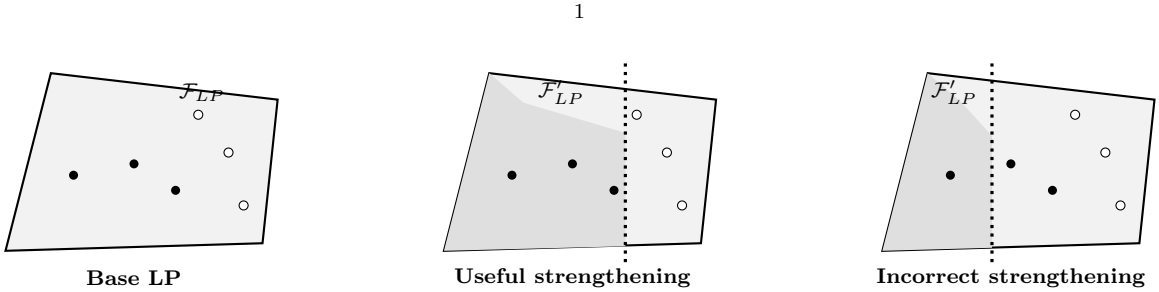


Figure 3.2: Strengthening the linear program with an additional constraint. Each panel shows the same feasible region and candidate solutions, with valid roots represented by filled black dots and invalid roots by unfilled white dots. The position of the additional constraint determines which solutions remain feasible. A suitable constraint eliminates fractional solutions while retaining all valid integer solutions, whereas an overly restrictive constraint also excludes valid integer solutions.

instances for which H exists have integral LP solutions. From $n = 5$ onwards, fractional solutions begin to occur, with their number increasing as n increases.

When H does not exist, two outcomes are observed. An LP infeasible instance has $\mathcal{F}_{LP} = \varnothing$, and therefore cannot contain a feasible integer solution. Alternatively, the LP may remain feasible but return a fractional solution, placing the solution in $\mathcal{F}_{LP} \setminus \mathcal{F}_{IP}$. The number of such fractional solutions also increases with n.

These results demonstrate that LP feasibility alone is insufficient to establish the existence of a graph root, since $\mathcal{F}_{LP}$ may contain both integral and fractional solutions. This is to be expected as LPs are solvable in polynomial time, but **SQUARE** is **NP**-complete. Fractional solutions arise because the linear relaxation weakens the integrality restrictions of the integer program, allowing variables that are binary in the integer formulation to take values in the continuous interval $[0, 1]$.

The following examines which properties of the integer formulation are preserved under the linear relaxation and identifies where fractional solutions can lose their interpretation as graph roots. Recall Equations (2.16)–(2.18), which impose $y_{ikj} = x_{ik}x_{kj}$. When x is binary, the construction y is binary as well; however, this may cause issues when x is fractional.

Recall Equation (2.12), where $z_{ii} = \sum_{k=1}^{n} y_{iki} - \sum_{k=1}^{n} x_{ik} + x_{ii}$ can be simplified to $z_{ii} = \sum_{k=1}^{n} x_{ik}^2 - \sum_{k=1}^{n} x_{ik}$ and is equal to 0 when x is binary. When x_{ik} is fractional, $x_{ik}^2 \leq x_{ik}$, such that $z_{ii} =$

$\sum_{k=1}^{n}(x_{ik}^2 - x_{ik}) \leq 0$. As $z \in \mathbb{Z}_{\geq} 0$, this gives that $zii = 0$ as required, and no additional modifications are required.

However, recall Equation (2.13), where $z_{ij} = \sum_{k=1}^{n} x_{ik}x_{kj} + x_{ij}$. When $z_{ij} = 0$, the equation follows the standard definition of graph squares, such that $x_{ij} = 0$ and, for all intermediary vertices k, at least one of x_{ik} or x_{kj} must be equal to 0. However, when $z_{ij} \geq 1$, x_{ij} and $x_{ik}x_{kj}$ may satisfy the constraints without giving a valid integral path in the root. Thus, the linear relaxation may remain feasible even when no graph root exists.

The following proposition establishes that whenever a graph root exists, the linear relaxation will never return an infeasible solution.

Proposition 1. *If G has a square root H, then $\mathcal{LP}(G)$ is feasible.*

Proof. Suppose that G has a square root H. By Theorem 5, the adjacency matrix A_H gives a feasible solution to $\mathcal{IP}(G)$. Since $\mathcal{LP}(G)$ is obtained by relaxing the integrality constraints of $\mathcal{IP}(G)$, every feasible solution of $\mathcal{IP}(G)$ is also feasible for $\mathcal{LP}(G)$. Hence $\mathcal{LP}(G)$ is feasible. □

3.2 Min Linear Program

Consider the replacement of Equation 2.8 with an objective function that minimizes the total weights on all edges to the linear relaxation defined in Section 3.1

$$\min \sum_{i=1}^{n}\sum_{j=1}^{n} x_{ij}. \tag{3.4}$$

The construction of the linear program for G using Equations (3.1)–(3.3), Equations (2.12)–(2.22) together with Equation (3.4) is denoted as $\min \mathcal{LP}(G)$.

This relaxation however yields more fractional solutions, as illustrated in Table 3.2.

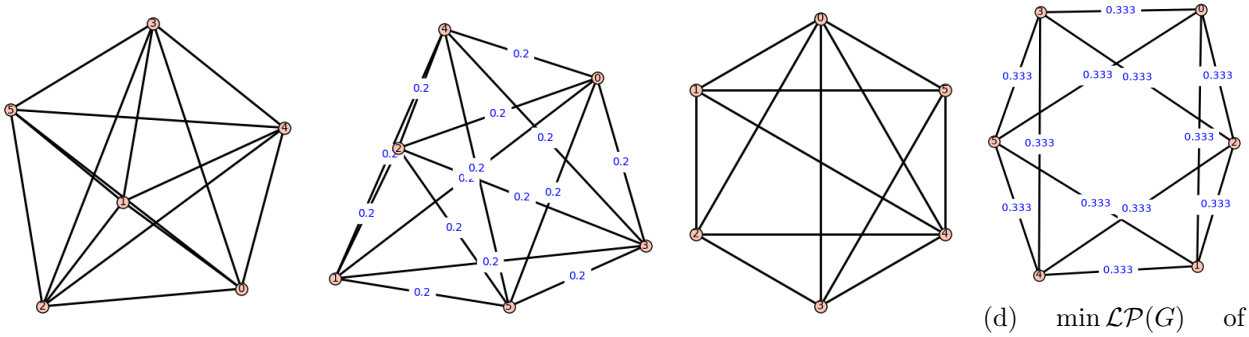
n	H exists		H does not exist	
	LP integral solution	LP fractional solution	LP infeasible	LP fractional solution
3	2	1	1	0
4	3	3	5	0
5	3	8	20	3
6	4	24	107	21
7	5	72	767	200
8	7	300	9380	2659

Table 3.2: LP outcomes for different values of n on the linear program with min objective function.

The behaviour of the weights observed in Figure 3.3 motivates the following proposition.

Proposition 2. *Let G be a graph. For every edge $uv \in E_G$, let t_{uv} be the number of vertices in a smallest maximal clique of G containing uv. Among solutions in which each edge variable is assigned the value determined by this rule, the unique minimizer of $\min \mathcal{LP}(G)$ is given by*

$$x_{uv} = \frac{1}{t_{uv} - 1}.$$



(a) K_6. (b) $\min \mathcal{LP}(G)$ of K_6. (c) $K_6 \backslash \{13, 25\}$. (d) $\min \mathcal{LP}(G)$ of $K_6 \backslash \{13, 25\}$.

Figure 3.3: For the square graph K_6, the result of $\min \mathcal{LP}(G)$ is completely fractional with all edges having weight $\frac{1}{5}$. For the square graph $K_6 \backslash \{13, 25\}$, the result of $\min \mathcal{LP}(G)$ is completely fractional with all edges having weight $\frac{1}{3}$

Proof. Let C be a maximal clique of G with $|C| = k$.

If $k = 2$, then C consists of an isolated edge uv. Hence

$$x_{uv} = 1 = \frac{1}{t_{uv} - 1}.$$

Now suppose $k \geq 3$. Since every pair of distinct vertices in C is adjacent, the indicator constraints Equations (2.14)–(2.15), together with the product constraints, imply that

$$\sum_{j \in V_C \backslash \{i\}} x_{ij} \geq 1 \quad \text{for every } i \in V_C.$$

If all edges of C receive the same weight t, this gives $(k-1)t \geq 1$. Since the contribution of the clique edges to the objective is $\binom{k}{2} t$, it is minimized uniquely at

$$x_{ij} = \frac{1}{k-1} \quad \text{for every } ij \in E_C. \tag{3.5}$$

Finally, suppose an edge ij belongs to multiple maximal cliques $C_1, \ldots, C_r$, of orders $k_1, \ldots, k_r$. Its weight must satisfy the lower bound imposed by each clique, giving

$$x_{ij} \geq \max_{1 \leq s \leq r} \frac{1}{k_s - 1} = \frac{1}{t_{ij} - 1},$$

where $t_{ij} = \min_{1 \leq s \leq r} k_s$. Thus, the smallest maximal clique containing ij determines the strongest lower bound on its weight. Consider the assignment $x_{uv} = \dfrac{1}{t_{uv} - 1}$ for all $uv \in E_G$. If uv lies in a maximal clique C of order k, then $t_{uv} \leq k$, so

$$x_{uv} = \frac{1}{t_{uv} - 1} \geq \frac{1}{k-1}.$$

Consequently, for every $i \in V_C$ $\displaystyle\sum_{j \in V_C \backslash \{i\}} x_{ij} \geq (k-1)\frac{1}{k-1} = 1$. Hence, the assignment satisfies Equation (3.5).

Therefore, among assignments satisfying the equal-weight requirement on every maximal clique, the minimum weights are

$$x_{uv} = \frac{1}{t_{uv} - 1} \quad \text{for all } uv \in E_G,$$

where t_{uv} is the order of a smallest maximal clique containing uv. □

Motivated by Proposition 2, we conjecture that $\min \mathcal{LP}(G)$ always returns a symmetric optimal solution, assigning equal weights to edges whenever possible.

Conjecture 1 (Fractional Cliques in $\min \mathcal{LP}(G)$)**.** *Let G be a graph with $n \geq 3$ vertices. For every edge $uv \in E_G$, let t_{uv} be the number of vertices in a smallest maximal clique of G containing uv. Then $\min \mathcal{LP}(G)$ returns the solution given by*

$$x_{uv} = \frac{1}{t_{uv} - 1} \quad \textit{for all } uv \in E_G.$$

3.3 Max Linear Program

Consider the replacement of Equation 2.8 with an objective function that maximizes the total weights on all edges to the linear relaxation defined by Equations (2.12)–(3.3) in Section 3.1

$$\max \sum_{i=1}^{n} \sum_{j=1}^{n} x_{ij}. \tag{3.6}$$

The construction of the linear program on the arbitrary graph G using Equations (3.1)–(3.3), Equations (2.12)–(2.22) together with Equation (3.6) is denoted as $\max \mathcal{LP}(G)$.

From Table 3.3, this objective encourages solutions that have fewer fractional edges and therefore tends to produce relaxations that are integral. In particular, maximizing the total edge weight pushes the variables closer to 1, which promotes direct edges and reduces the need for paths of length 3 with fractional weights to satisfy Equation (2.13). As a result, the relaxation is more likely to yield integral solutions.

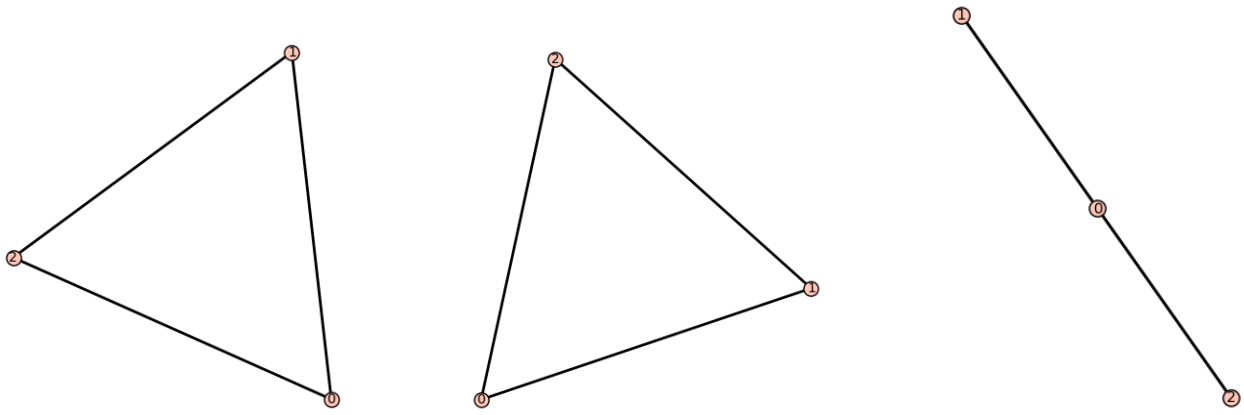
n	H exists		H does not exist	
	LP integral solution	LP fractional solution	LP infeasible	LP fractional solution
3	3	0	1	0
4	6	0	5	0
5	9	2	20	3
6	18	10	107	21
7	38	39	767	200
8	94	213	9380	2659

Table 3.3: LP outcomes for different values of n on the linear program with max objective function.

This choice of objective function is also useful when identifying solutions that are not unique as labelled graphs but are unique up to isomorphism. For example, the complete graph K_3 has four graph roots: K_3 itself and three path graphs on three vertices. The three path roots are distinct as labelled graphs but are all isomorphic to P_3. Figure (3.4) illustrates this situation.

Without an objective function that breaks this symmetry, the three labelled path roots are indistinguishable to the linear program, and their symmetric combination can give rise to a fractional solution rather than selecting a single path. In contrast, the maximization objective function

breaks this symmetry by assigning different objective values to the possible labelled edge selections. Consequently, the single complete graph K_3 is selected as the unique optimal solution.



(a) K_3. (b) K_3 root of K_3. (c) A path root of K_3.

Figure 3.4: The graph K_3 and two of its square roots. Notice that there are multiple possible isomorphic P_3 roots.

3.3.1 Fundamental Properties

We now establish several fundamental properties of feasible solutions to $\mathcal{LP}(G)$ that follow directly from the formulation. These properties will be used throughout the subsequent analysis of the linear relaxation.

Lemma 11. *Let G and H be graphs such that $G = H^2$. Let $a, b \in V_G$ such that $ab \notin E_G$. For every feasible solution x of $\mathcal{LP}(G)$ and every $v \in V_G$ with $v \neq a, b$ we have $x_{ab} = y_{avb} = 0$.*

Proof. Using Equations (2.14)–(2.15) if $g_{ab} = 0$, then $z_{ab} \leq 0$ together with $z_{ab} \geq= 0$ implies that $z_{ab} = 0$. Then Equation (2.13) gives that $0 = \sum_{v=1}^{n} y_{avb} + x_{ab}$ which only holds when each variable is equal to 0 since variables of the form x_{ab}, y_{avb} are non-negative. This gives that $x_{ab} = 0$ and $y_{avb} = 0$ for all vertices $v \in V \backslash \{a, b\}$. □

Lemma 12. *Let G and H be graphs such that $G = H^2$. Let $a, b \in V_G$ such that $ab \notin E_G$. For every feasible solution x of $\mathcal{LP}(G)$ and every $v \in V_G$ with $v \neq a, b$ we have*

$$x_{av} + x_{vb} \leq 1.$$

Proof. By Lemma (11), $y_{avb} = 0$. By Equation (2.15), $0 = y_{avb} \geq x_{av} + x_{vb} - 1$. Rearranging yields $x_{av} + x_{vb} \leq 1$, as required. □

Lemma 13. *Let G and H be graphs such that $G = H^2$. Let $a, b, v \in V_G$ such that $ab \notin E_G$. If a feasible solution x of $\mathcal{LP}(G)$ satisfies $x_{av} = 1$, then $x_{vb} = 0$.*

Proof. Since $ab \notin E_G$, Lemma (12) gives $x_{av} + x_{vb} \leq 1$. Since $x_{av} = 1$, it follows that $1 + x_{vb} \leq 1$, and hence $x_{vb} \leq 0$. By the non-negativity of Equation (3.1), $x_{vb} \geq 0$, and therefore $x_{vb} = 0$. □

Lemma 14 (0 Path Propagation)**.** *Let G and H be graphs such that $G = H^2$. Let $a, i, j, b \in V_G$ such that a, i, j, b form a shortest induced path in H between a and b. For every feasible solution x of $\mathcal{LP}(G)$,*

$$x_{ai} = 1 \implies x_{ib} = 0, \quad x_{ia} = 1 \implies x_{bi} = 0, \quad x_{jb} = 1 \implies x_{aj} = 0, \quad x_{bj} = 1 \implies x_{ja} = 0.$$

Proof. Since a, i, j, b form shortest path in H, we have $d_H(a, b) = 3$ and therefore $ab \notin E_G$. By Lemma (13), setting $x_{ai} = 1$ gives $x_{ib} = 0$. Similarly, setting $x_{jb} = 1$ gives $x_{aj} = 0$. The remaining two implications follow by symmetry. □

Lemma 15 (1-Path Propagation)**.** *Let G be a graph and let x be a feasible solution of $\mathcal{LP}(G)$. Suppose that $u - v - a_1$ form a shortest induced path in H between u and a and that $x_{uv} = 1$. Let $A = \{a_1, \ldots, a_r\}$ and $B = \{b_1, \ldots, b_s\}$ be such that every vertex $k \in V_G \setminus \{u, v\}$ satisfies $k \in A \cup B$, and suppose that*

$$x_{um} = 0 \text{ for every } m \in A, \quad g_{un} = 0 \text{ for every } n \in B,$$

and $g_{sa_1} = 0$ for the vertex s preceding u on the shortest path to a_1. Then

$$x_{va_1} = 1.$$

Proof. Without loss of generality consider $a_1 \in A$. Since $ua_1 \in E_G$, have $g_{ua_1} = 1$, and hence Equation (2.15) gives $z_{ua_1} \geq 1$. Since $x_{ua_1} = 0$, Equation (2.13) gives

$$1 \leq z_{ua_1} = x_{ua_1} + \sum_{k \in V_G} y_{uka_1} = \sum_{k \in V_G} y_{uka_1}.$$

The terms corresponding to $k = s$ and $k = u$ are zero since $g_{sa_1} = g_{uu} = 0$, and hence Lemma (11) gives $x_{sa_1} = x_{uu} = 0$. Therefore, by Equations (2.16)–(2.18),

$$y_{usa_1} = y_{uua_1} = 0.$$

For every $m \in A$, we have $x_{um} = 0$, and hence Equation (2.16) gives

$$y_{uma_1} = 0.$$

Similarly, for every $n \in B$, we have $g_{un} = 0$, and therefore Lemma (11) gives $x_{un} = 0$. Thus Equation (2.16) gives

$$y_{una_1} = 0.$$

Thus all terms in the sum are zero except possibly y_{uva_1}, giving

$$1 \leq y_{uva_1}.$$

Since $x_{uv} = 1$, Equation (2.16) gives $y_{uva_1} \leq x_{uv} = 1$, while Equation (2.17) gives $y_{uva_1} \leq x_{va_1}$. Hence $1 \leq y_{uva_1} \leq x_{va_1} \leq 1$, and therefore

$$x_{va_1} = 1.$$

□

3.4 Leaf Fixing

In this section, we show that every tree contains one of two structural configurations that can be identified from its square G. These configurations allow us to identify a leaf, or a group of leaves, and thereby recover a path $s - u - v$ in H, where s is a leaf. Once such a path is identified, we can fix the corresponding edge variables and use the resulting constraints to propagate the assignments through the LP.

Definition 3.4.1 (Sparse Pendant)**.** *Let H be a graph. A **sparse pendant** is a path $s - u - v$ in H such that s is a leaf and u has degree 2.*

Definition 3.4.2 (Pendant Fork)**.** *Let H be a graph. A **pendant fork** is the induced subgraph of H on four vertices s_1, s_2, u, v, where s_1 and s_2 are leaves adjacent to u, and every neighbour of u other than v is a leaf.*

Lemma 16. *Every tree with at least 3 vertices contains either a sparse pendant or a pendant fork subgraph.*

Proof. Let T be a rooted tree with $|V_T| \geq 3$, and root T at an arbitrary vertex. Since T is finite, there exists a leaf, say s, of maximum depth. Let u be the parent of s. Every child of u is a leaf, since if there was some child w of u that is not a leaf, then w would have a child and hence there would exist a vertex at depth strictly greater than that of s, contradicting the choice of s.

If u is not the root, let v denote its parent. Every neighbour of u other than v is a child of u, and hence is a leaf. If u has exactly one such leaf neighbour, say s, then u has exactly two neighbours, namely s and v, and hence $\deg_T(u) = 2$. Therefore, the induced subgraph on

$$\{s, u, v\}$$

forms a pendant subgraph.

Otherwise u has at least two leaf neighbours s_1, s_2. Choose another neighbour v of u distinct from s_1, s_2. If u is not the root, choose v as its parent. Since v is distinct from s_1 and s_2, have $\{s_1, s_2, v\} \subseteq N(u)$, and $\deg_T(u) \geq 3$. Hence the induced subgraph on

$$\{s_1, s_2, u, v\}$$

forms a pendant fork subgraph.

If u is the root, then all neighbours of u are children and hence are leaves. Since $|V_T| \geq 3$, u has at least two neighbours. If it has exactly two neighbours, the induced subgraph on these three vertices forms a sparse pendant. Otherwise, choose any three distinct children s_1, s_2, v of u. Then $\{s_1, s_2, v\} \subseteq N_T(u)$, and the induced subgraph on

$$\{s_1, s_2, u, v\}$$

forms a pendant fork subgraph. □

Using this lemma, every tree contains either a pendant or a pendant fork subgraph. Consequently, whenever G admits a tree root, we may identify one of these two configurations in the square and use this identification to derive valid fixings for the variables x_{ij}.

3.4.1 Single Triangular Decomposition

We first consider how to identify the sparse pendant described in Definition 3.4.1 from G. For a vertex $s \in V$, define

$$T(s) = \big\{\{u, v\} \subseteq V \setminus \{s\} | (s, u) \in E,\ (s, v) \in E,\ (u, v) \in E\big\}$$

to be the set of K_3s in G that contain s. If $|T(s)| = 1$ where $T(s) = \{\{u, v\}\}$ then the vertex s exists in a unique K_3 of G. When the neighbours have different degree $\deg_G(u) \neq \deg_G(v)$, then the vertex with smaller degree must be the stem vertex in the square root. This is assigned deterministically for a single triangle s using

$$x_{sa} = x_{as} = x_{ab} = x_{ba} = 1, \quad \text{where } (a, b) = \begin{cases} (u, v), & \deg_G(u) \leq \deg_G(v), \\ (v, u), & \deg_G(u) > \deg_G(v). \end{cases} \tag{3.7}$$

If $\deg_G(u) = \deg_G(v)$, then either u, v can be chosen since they are connected to a clique.

Proposition 3 (Single Triangle Decomposition Interchangeability)**.** *Let G be a graph containing a triangle on vertices s, u, v, such that $\{s, u, v\}$ is the unique maximal clique containing s and $\deg_G(u) = \deg_G(v) = d \geq 2$. If G admits a graph root, then G admits a tree root T such that $su, uv \in E_T$.*

Proof. Since s lies in a single maximal clique in G, in any tree root T, the vertex s must be a leaf, with exactly one of u or v being the internal vertex adjacent to the remainder of the graph. Assume without loss of generality that u is this internal vertex. Since v has no other connections in the maximal clique besides s and u, any vertex adjacent to v in G other than s must also be adjacent to u in G. Hence

$$N_G(v) \setminus \{s\} \subseteq N_G(u) \setminus \{s\}.$$

Since $\deg_G(u) = \deg_G(v)$, the two sets have equal cardinality, and therefore

$$N_G(u) \setminus \{s\} = N_G(v) \setminus \{s\}.$$

Thus u and v have identical neighbourhoods outside s and may be interchanged without changing the square graph G. Consequently, if G admits a square root, we may choose u as the internal vertex such that $su, uv \in E_T$. □

3.4.2 Tetrahedron Decomposition

We now consider how to identify the pendant fork described in Definition 3.4.2 from G.

A natural extension of Section 3.4.1 can be achieved by considering cliques of size greater than three in order to categorize tree structure whereby an internal vertex is connected to more than one leaf. Whilst finding a clique of size $\geq k$ for fixed k is polynomial, with complexity of order $O(n^k)$ by simply checking all tuples of size k, the decision problem of whether a graph G contains a clique of size k, as well as the problem of finding a largest clique, is **NP**-complete. Thus, it is necessary to consider only cliques of fixed size 4, for which the problem of determining their existence has polynomial complexity $O(n^4)$. This however is sufficient for classifying any pendant fork, since any branching vertex with exactly two leaves will generate a K_4 in the square, whilst a branching vertex with more than two leaves will generate multiple K_4s of which one can be chosen by arbitrarily selecting any two leaves.

Consider vertices $s_1, s_2, u, v \in V$ such that $K_4 = \{s_1, s_2, u, v\}$ induces a K_4 in G and for each leaf s_1, s_2 of u their neighbourhoods are equivalent and give the common neighbourhood $N_L = N_G(s_1) \setminus K_4 = N_G(s_2) \setminus K_4$. If

$$\big(N_G(s_1) \setminus K_4\big) = \big(N_G(s_2) \setminus K_4\big) \subset \big(N_G(u) \setminus K_4\big) \subseteq \big(N_G(v) \setminus K_4\big), \tag{3.8}$$

and if every vertex in the common neighbourhood of each leaf has no neighbours outside the tetrahedron and that neighbourhood itself,

$$N_G(w) \subseteq K_4 \cup N_L \quad \forall w \in N_L, \tag{3.9}$$

then u is designated as the internal vertex, v as the external vertex, and s_1, s_2 as the leaf vertices of the tetrahedron.

Under Equations (3.8) and (3.9), the vertices s_1 and s_2 correspond to leaves adjacent to a common internal vertex u, whilst v acts as the connecting vertex between u and the remainder of the graph. The vertices with identical minimal neighbourhood structure correspond to leaves adjacent to a common internal vertex u, whilst the vertex v, having maximal neighbourhood containment, acts as the connecting vertex between u and the remainder of the graph.

For $n = 4$, the vertex u is a binary branching vertex contained in a unique K_4. In particular, when $n = 5$, the branching vertex u has three leaf children s_1, s_2, s_3, producing $\binom{3}{2} = 3$ distinct K_4s

$$\{s_1, s_2, u, v\}, \quad \{s_1, s_3, u, v\}, \quad \{s_2, s_3, u, v\}.$$

More generally, suppose u has k leaf children $s_1, \ldots, s_k$, and that these vertices together with v induce a K_{k+2}. Then every pair of leaves (s_i, s_j) determines a distinct K_4 $\{s_i, s_j, u, v\}$ where the total number of such K_4s generated by this branching configuration is $\binom{k}{2}$. Suppose that s_1, s_2 are chosen without loss of generality, then if the K_4 satisfies Equation (3.8) and Equation (3.9) have

$$x_{s_1u} = x_{us_1} = x_{s_2u} = x_{us_2} = x_{uv} = x_{vu} = 1. \tag{3.10}$$

Proposition 4 (Tetrahedron Decomposition Interchangeability)**.** *Let G be a graph that admits a graph root, and let s_1, s_2, u, v form a K_4 in square G. If Equation* (3.8) *and Equation* (3.9) *are satisfied and*

$$N_G(u) \setminus K_4 = N_G(v) \setminus K_4,$$

then G admits a tree root T such that $s_1u, s_2u, uv \in E_T$.

Proof. Since s_1, s_2, u, v form a K_4 in G and

$$N_G(u) \setminus K_4 = N_G(v) \setminus K_4,$$

the vertices u and v have identical neighbourhoods outside K_4. Moreover, since u and v are both adjacent to every vertex of K_4, it follows that

$$N_G(u) = N_G(v).$$

Thus, exchanging u and v is an automorphism of G. Hence, if the root of G is a tree root T and has v as the internal vertex of the tetrahedral configuration, applying this automorphism allows the roles of u and v to be exchanged without changing the square graph. Therefore, without loss of generality may assume that u is the internal vertex of the tetrahedral configuration in the root G and such that $s_1u, s_2u, uv \in E_T$ □

3.5 Unicliqual Vertex Linear Program

The key idea is to strengthen the formulation by adding constraints corresponding to edges that are known to occur in a graph root whenever a root exists. Using, Lemma (16), the formulations developed in Subsection 3.4.1–3.4.2 can be combined into one succinct linear program by first identifying a leaf-steam structure involving three vertices s, u, v. In particular, if a graph root exists, the structural properties of the graph guarantee that there is a root containing the identified edges. We may therefore safely fix the corresponding variables to 1, thereby restricting the formulation to solutions containing these edges without excluding any graph root.

If s is a leaf, u is its corresponding stem, and there exists $v \in N_G[u] \setminus \{u\}$ such that

$$N_G[s] \subset N_G[u] \subseteq N_G[v],$$

with all vertices in $N_G[s] \setminus \{u, v\}$ having the same closed neighbourhood as s, then the corresponding root edges can be fixed by imposing

$$x_{su} = x_{us} = x_{uv} = x_{vu} = 1. \tag{3.11}$$

Proposition 5 (Unicliqual Vertex Interchangeability)**.** *Let s, u, v form a K_3 in square G, where s is unicliqual. If $N_G[u] = N_G[v]$ then u and v may be chosen arbitrarily as the corresponding vertex in the tree root T without loss of generality.*

Proof. Since s is unicliqual, the maximal clique containing s is uniquely determined. If this clique satisfies the conditions of Proposition 3, then that proposition permits either u or v to be chosen as the corresponding vertex, since $N_G[u] = N_G[v]$ gives $\deg_G(u) = \deg_G(v)$. If the configuration instead satisfies the conditions of Proposition 4, then the equality $N_G[u] = N_G[v]$ gives

$$N_G(u) \setminus K_4 = N_G(v) \setminus K_4,$$

so either u or v may be chosen in the tetrahedron fixing. Hence, in either case, u and v are interchangeable, and either may be chosen without loss of generality. □

It remains to verify that whenever a root H of G exists, the addition of the constraint Equation (3.11) preserves the feasibility of the linear program.

Proposition 6 (Validity of Unicliqual Vertex Fix)**.** *If a root H of G exists, then* $\max \mathcal{UVLP}(G)$ *is feasible.*

Proof. Suppose that a root H of G exists. By Proposition 1, $\mathcal{LP}(G)$ is feasible. It remains to show that there exists a root whose adjacency matrix satisfies Equation (3.11).

Let s, u, v be the vertices identified by the unicliqual vertex fixing. By Proposition 5, if s, u, v form the corresponding K_3 and $N_G[u] = N_G[v]$, then u and v may be chosen arbitrarily as the corresponding vertex in the tree root without loss of generality. Thus, if the given root H does not satisfy Equation (3.11), the roles of u and v may be interchanged to obtain another root H' of G for which

$$x_{su} = x_{us} = x_{uv} = x_{vu} = 1.$$

Hence the adjacency matrix of H' is a feasible solution of $\mathcal{IP}(G)$ that satisfies Equation (3.11). Since every feasible solution of $\mathcal{IP}(G)$ is feasible for $\mathcal{LP}(G)$, this solution is also feasible for $\max \mathcal{UVLP}(G)$. Therefore, $\max \mathcal{UVLP}(G)$ is feasible whenever a root H of G exists. □

n	H exists		H does not exist	
	LP integral solution	LP fractional solution	LP infeasible	LP fractional solution
3	3	0	1	0
4	6	0	5	0
5	10	1	21	2
6	22	6	117	11
7	51	26	852	115
8	148	159	10271	1768

Table 3.4: Outcomes of the unicliqual vertex linear program for graphs of different orders of n.

3.6 Tree Proof

We first show that the unicliqual vertex linear program will always fix a sparse pendant or pendant fork.

Lemma 17. *Let G and T be graphs such that $G = T^2$ and T is a tree.* $\max \mathcal{UVLP}(G)$ *always fixes a sparse pendant or pendant fork subgraph of T.*

Proof. By Lemma 16, T either contains a sparse pendant or a pendant fork subgraph. Let s be a leaf of this subgraph and let u be its stem. Since s is a leaf of T, every vertex of $N_G[s]$ is either s, u, the neighbours of u in T, or a vertex at distance two from s through u. Hence

$$N_G[s] \subset N_G[u].$$

Moreover, if v is the neighbour of u in T that connects the sparse pendant or pendant fork to the remainder of T, then every vertex adjacent to u in T is also within distance two of v in T. This gives

$$N_G[u] \subseteq N_G[v].$$

In the pendant fork case, the other leaf s' of the fork satisfies $N_G[s'] = N_G[s]$, since both s and s' are leaves adjacent to the same vertex u. Thus the vertices in $N_G[s] \setminus \{u, v\}$ have the same closed neighbourhood as s.

Therefore the vertices s, u, v satisfy the conditions for Equation (3.11), which imposes

$$x_{su} = x_{us} = x_{uv} = x_{vu} = 1.$$

□

Lemma 18 (Propagation)**.** *Let G and T be graphs such that $G = T^2$ and T is a tree. Let $s, u, v \in V_T$ such that s is a leaf, u is its stem, and v is the external vertex. Let $a_1, a_2, \ldots a_r$ be the set of neighbours of v in T. In every feasible solution x of $\mathcal{UVLP}$,*

$$x_{va_i} = x_{a_iv} = 1 \text{ for every } i \in \{1, \ldots, r\}.$$

Proof. Since s is a leaf and u is a stem of T, the path $s{-}u{-}v$ is contained in T. Let $A = \{a_1, \ldots, a_r\}$ denote the vertices adjacent to v in T other than u, and let $B = \{b_1, \ldots, b_s\}$ denote the vertices adjacent to a vertex in A other than v. For each $a_i \in A$, the vertices s, u, v, a_i form a P_4 in T. Since $x_{su} = 1$ and there exists no cycles in T, Lemma (14) gives

$$x_{ua_i} = 0 \quad \text{for all } a_i \in A.$$

Using Lemma (15) This process can be repeated for each $a_i \in A$, giving $x_{va_i} = 1$ for all $i \in \{1, \ldots, r\}$. Thus the edges $uv, va_1, \ldots, va_r$ are identified in the root, with u acting as a leaf, v as its stem, and each a_i as the next vertex along the tree. □

Theorem 6. *Let G and T be graphs such that $G = T^2$ and T is a tree. Then $\mathcal{UVLP}(G)$ returns an integral solution.*

Proof. Suppose that $G = T^2$ for some tree T. We consider two cases.

First, suppose that T is a star. Then G is a complete graph and the maximum objective therefore admits an integral solution that is the complete graph itself. Unicliqual fixing is not applied and unnecessary.

Now suppose that T is not a star. By Lemma 17, the unicliqual vertex fixing identifies either a sparse pendant or a pendant fork in T.

In the sparse pendant case, the fixing identifies a leaf s, its stem u, and the external vertex v. Equation (3.11) imposes

$$x_{su} = x_{us} = x_{uv} = x_{vu} = 1$$

in $\mathcal{UVLP}(G)$.

In the pendant fork case, the fixing identifies leaves s_1, s_2 with common parent u, together with the external vertex v adjacent to u. The remaining leaves of u need not be considered individually, as they form a clique with the other leaves and u by the maximum objective function. Selecting one of the s_1 leaves as s likewise imposes

$$x_{s_1u} = x_{us_1} = x_{uv} = x_{vu} = 1$$

in $\mathcal{UVLP}(G)$.

Let $N_T(v) \setminus \{u\} = \{a_1, \ldots, a_r\}$. Hence the hypotheses of Lemma 18 are satisfied and the path propagation lemma implies

$$x_{va_i} = x_{a_iv} = 1 \text{ for every } i \in \{1, \ldots, r\},$$

for every $a_i \in N_T(v) \setminus \{u\}$.

We prove by induction on the number of edges of the reconstructed subtree that repeated application of the path propagation lemma assigns value 1 to every edge of T.

For the base case, the reconstructed subtree consists of the edges initially identified by the unicliqual vertex fixing and the first application of path propagation. These edges have integral assignments by Equation (3.11) and Lemma 18.

For the inductive step, suppose that a connected subtree T' of T has been reconstructed and that every edge of T' has been assigned its integral value. If $T' \neq T$, since T is connected, there exists an edge

$$ab \in E_T \setminus E_{T'}$$

with $a \in V_{T'}$ and $b \notin V_{T'}$. Since T' contains at least one path of length two, we may arbitrarily label three consecutive vertices of such a path as s, u, v. The vertex a is then the already reconstructed vertex adjacent to the unlabelled vertex b. Relabelling the vertices in this local configuration in this way, the hypotheses of Lemma 18 are satisfied. Hence path propagation fixes

$$x_{ab} = x_{ba} = 1.$$

Thus T' can be extended to the larger connected subtree $T'' = T' + ab$, whose edges all have integral assignments.

Thus, by the inductive argument, whenever the reconstructed subtree is a proper subtree of T, it can be extended by at least one additional edge of T, with all edges of the resulting subtree assigned integral values. Since T is finite, the induction eventually reconstructs all of T and assigns an integral value to every edge. Hence the resulting solution is integral and corresponds to the tree root T of G. Therefore, $\max \mathcal{UVLP}(G)$ returns an integral solution. □

Chapter 4

Further Work

In this chapter, we discuss additional work, further concepts, and interesting ideas arising from structural properties that the square graph intrinsically possesses.

Previous methods for solving specific classes of roots have been developed by augmenting the linear programming framework with additional constraints and objective functions. Nevertheless, integrating known polynomial-time solutions for trees into an overall framework, while maintaining the internal consistency that valid square graphs must always correspond to feasible instances, does not appear to justify the use of a comparatively complex polynomial-time framework such as the interior-point method.

It is, however, possible to construct stronger exact algorithms by performing a polynomial number of runs of the linear program. Each run can be used to identify additional structural information, fix variables, or further restrict the possible solutions before solving the resulting instance again. This may provide a useful mechanism through which improvements to the base linear program can have a much greater effect than the improvement obtained from a single LP solution alone. In particular, a stronger base program provides more integral variables and structural information, which can then be propagated through subsequent runs of the LP relaxation.

The following sections illustrate several directions in which this can be explored.

4.1 Vertex Deletion

We can use an LP to find the set of all graphs on $n \leq 5$ vertices that admit a root. From Table (3.4) all square graphs can be completely categorized for small n, such that, whenever a root H exists, an integral solution is obtained, while whenever no root H exists, the linear program is infeasible.

In particular, consider arbitrarily removing a vertex from G and applying the same linear program to the resulting subgraph. If the subgraph has an integral solution, this provides structural information about a possible root on the reduced vertex set. We may then fix a partial solution using vertices whose root edges are not generated by a P_3 whose middle vertex is the removed vertex. The natural question is whether this information can subsequently be used to constrain the original graph when the deleted vertex is reintroduced.

More specifically, if deleting a vertex produces a graph for which the linear program has an integral solution, it is worth investigating whether the corresponding solution can be extended to the original graph and, if so, whether the possible extensions can be restricted to a small number of cases. If such extensions can be characterized, vertex deletion provides a mechanism through which information obtained from smaller instances can be propagated back to the original graph.

Let G^- be the subgraph of G defined by $V^- = V - v, E_{G^-} = \{E_G \backslash uv \mid u \in V^-\}$. If G^- admits

a valid square root, denote it by H^-. Let H be a square root of G satisfying $H^2 = G$, and let H_- be the subgraph of H obtained by deleting v, defined by $V^-, \{E_H \backslash uv \mid u \in V^-\}$.

In general, H_- need not be a square root of G^-, since the deletion of v from H may remove copies of P_3 between vertices in V^- which results in $(H_-)^2 \neq G^-$. Nevertheless, an integral solution of the LP for G^- provides a candidate set of root edges on V^- which can be used to refine the LP for G.

Algorithm 2 Vertex-Deletion Linear Program

Input: A graph G
Output: A LP model (m, x)
Construct and solve $\mathcal{LP}(G)$.
if $\mathcal{LP}(G)$ is integral **then**
    return (m, x)
end if
Set $F = \left\{ \{a, b\} \in \binom{V_G}{2} : x_{ab} = x_{ba} = 1 \right\}$.
Compute $d_F(u) = |\{w \in N_G(u) : \{u, w\} \notin F\}|$ for each $u \in V_G$.
Choose $v \in \arg\min_{u \in V_G} d_F(u)$ and set $V^- = V_G \setminus \{v\}$.
Construct $G^- = G[V^-]$ and inherit the fixing $F^- = \left\{\{a, b\} \in F : a, b \in V^-\right\}$.
Construct and solve $\mathcal{LP}(G^-) + F^-$.
if $\mathcal{LP}(G^-) + F^-$ is integral **then**
    Construct H^- from its integral solution.
else
    return (m, x)
end if
Construct a new copy of $\mathcal{LP}(G)$ and impose $x_{ab} = x_{ba} = 1$ for every $\{a, b\} \in F$ and every $ab \in E_{H^-}$.
Solve the modified LP.
if the modified LP is infeasible **then**
    return (m, x)
else
    return the modified LP solution (m, x)
end if

n	H exists		H does not exist	
	LP integral solution	LP fractional solution	LP infeasible	LP fractional solution
3	3	0	1	0
4	6	0	5	0
5	11	0	21	2
6	25	3	117	11
7	62	15	852	115
8	183	124	10271	1768

Table 4.1: Outcomes of the vertex deletion linear program using unicliqual vertex as the base for graphs of different orders of n.

The recursive algorithm works as follows.

i) Solve $\mathcal{LP}(G)$. If the LP is infeasible, the procedure terminates. If the LP is integral, the corresponding graph is retained as a candidate square root and the procedure terminates. Otherwise, let $F \subseteq \binom{V}{2}$ denote the set of unordered vertex pairs for which the solution satisfies $x_{ab} = x_{ba} = 1$.

ii) For each $u \in V$, define its unfixed degree with respect to F to be $d_F(u) = \{w \in N_G(u) : u, w \notin F\}$. Choose a vertex $v \in \arg\min_{u \in V}\{d_F(u)\}$ and construct $G^- = G - v$ by deleting v and all edges incident to v. Let $V^- = V \setminus v$ and let $F^- = a, b \in F, b \in V^-$ denote the fixing inherited by G^-.

iii) Solve $\mathcal{LP}(G^-) + F^-$, which is G^- with the additional fixing F^-.

   (a) If $\mathcal{LP}(G^-) + F^-$ is fractional, let H^- denote the corresponding root of F^-.

   (b) If $\mathcal{LP}(G^-) + F^-$ is infeasible, the candidate H^- is discarded.

   (c) If $\mathcal{LP}(G^-)+F^-$ is fractional, recursively apply the vertex deletion algorithm to solve $\mathcal{G}^-$. If the recursive deletion produces an integral solution, let H^- denote the corresponding root of G^-.

iv) If an integral candidate H^- has been obtained, construct $\mathcal{LP}(G \mid F, H^-)$ by taking $\mathcal{LP}(G)$ and imposing $x_{ab} = x_{ba} = 1$ for every $a, b \in F$ and every pair $a, b \in \binom{V^-}{2}$ such that $ab \in E_{H^-}$. No additional constraint is imposed on pairs for which $ab \notin E_{H^-}$, and the variables corresponding to pairs involving v remain unrestricted apart from the fixing induced by F.

v) Solve $\mathcal{LP}(G \mid F, H^-)$.

   (a) If $\mathcal{LP}(G \mid F, H^-)$ is feasible, the fixing obtained from H^- is consistent with the LP formulation for G, and the resulting solution is retained.

   (b) If $\mathcal{LP}(G \mid F, H^-)$ is infeasible, the candidate H^- is discarded. This does not imply that $\mathcal{LP}(G)$ is infeasible; it only shows that the candidate root structure H^- is incompatible with the current fixing for G.

vi) If no integral candidate H^- is obtained from $\mathcal{LP}(G^-) + F^-$ or from the recursive application to G^-, then no further fixing is obtained from the vertex deletion step. The current feasible solution of $\mathcal{LP}(G)$ is retained, and the procedure terminates.

vii) If an integral candidate H^- is obtained but $\mathcal{LP}(G \mid F, H^-)$ is infeasible, the fixing induced by H^- is discarded. The current feasible solution of $\mathcal{LP}(G)$ is retained, and the procedure terminates. This does not imply that $\mathcal{LP}(G)$ is infeasible; it only shows that the candidate root structure H^- is incompatible with the current fixing.

If the candidate H^- is discarded, instead of applying Step vi), consider the remaining square roots of G^-, taking one representative from each possible non-isomorphic solution. For each such root H', construct $\mathcal{LP}(G \mid F, H')$ by imposing $x_{ab} = x_{ba} = 1$ for every $a, b \in \binom{V^-}{2}$ such that $ab \in E_{H'}$, in addition to the fixing induced by F. If the resulting LP is feasible, the corresponding solution is retained.

The intuition behind this step is that the linear program only finds a single solution, while there may be many distinct, non-isomorphic solutions. Hence, although fixing a subgraph obtained from the LP may be useful, the particular subgraph selected by the LP may not correspond to the

n	H exists		H does not exist	
	LP integral solution	LP fractional solution	LP infeasible	LP fractional solution
3	3	0	1	0
4	6	0	5	0
5	11	0	21	2
6	25	3	117	11
7	62	15	852	115
8	194	113	10271	1768

Table 4.2: Outcomes of the recursive vertex deletion linear program using unicliqual vertex as the base for graphs of different orders of n.

subgraph required to continue the reconstruction of a square root. Resolving this issue would require a polynomial-time method for identifying an appropriate subgraph among the possible solutions, which is not provided by the current enumerative implementation. Hence, the resulting approach is not polynomial in its current form. Nevertheless, the approach shows very strong improvements in practice as can be seen in Table 4.3.

n	H exists		H does not exist	
	LP integral solution	LP fractional solution	LP infeasible	LP fractional solution
3	3	0	1	0
4	6	0	5	0
5	11	0	21	2
6	26	2	117	11
7	73	4	852	115
8	274	33	10271	1768

Table 4.3: Outcomes of the recursive vertex deletion linear program using unicliqual vertex as the base and enumerating all possible H^- roots for graphs of different orders of n.

4.2 Symbolic Sets

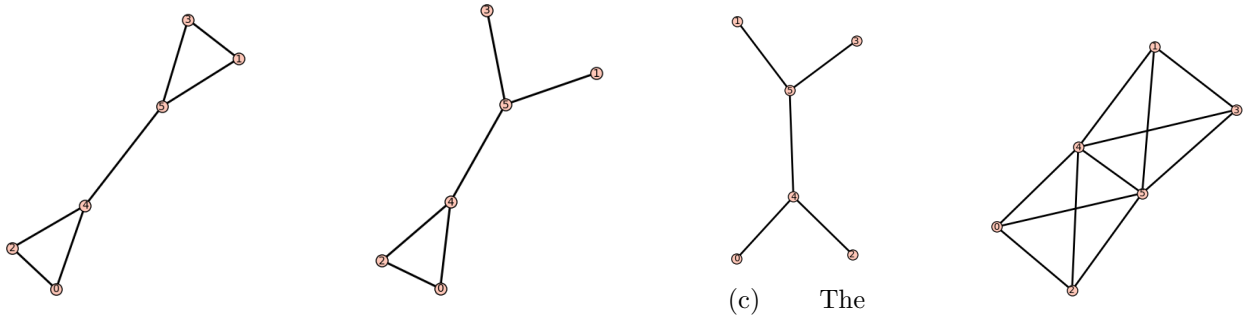
Lemma 12 gives structure into the feasible solution of the LP. Using the maximization objective function Lemma 12, if $g_{ij} = g_{jk} = 1G$ but $g_{ki} = 0$, $x_{ij} + x_{jk} \leq 1$.

If one could determine when this holds with equality $x_{ij} + x_{jk} = 1$, this would enable x_{ij}, x_{jk} to be easily categorized since if one is 0 then the other is necessarily 1.

Conjecture 2. *Let G and H be graphs such that $G = H^2$. Let $a, b \in V_G$ such that $ab \notin E_G$. For every feasible solution x of $\max \mathcal{LP}(G)$ and every $v \in V_G$ with $v \neq a, b$ we have*

$$x_{av} + x_{vb} = 1 \text{ or } x_{av} + x_{vb} = 0.$$

Consider Figure 4.1 where G_1 is the unique root with maximum objective value. Assigning $\alpha = x_{14}$ gives using Conjecture (2) that $1 - \alpha = x_{40}$ since $g_{01} = 0$. The same applies such that $1-\alpha = x_{42}$ and $\alpha = x_{43}$, whereby the existence of edges $g_{13} = g_{02} = 1$ prevent additional constraints being imposed upon α and ensure it is internally consistent. By symmetry can likewise assign $\beta = x_{15} = x_{35}$ and $1-\beta = x_{50} = x_{52}$. Setting $\alpha = 0$ and $\beta = 1$ thus gives $x_{40} = x_{42} = x_{15} = x_{35} = 1$



(a) The root graph G_1.

(b) The root graph G_2.

(c) The root graph G_3.

(d) The graph square $H = G_i^2$.

Figure 4.1: Three non-isomorphic square roots of the same six-vertex graph square.

and $x_{14} = x_{43} = x_{50} = x_{52} = 0$ which with the addition of $x_{45} = 1$ gives Figure 4.1c. In this example, both $\{(1,4),(4,0),(4,2),(4,3)\}$ and $\{(1,5),(3,5),(5,0),(5,2)\}$ are consistent sets.

To formalise this two-colouring structure, we work directly with subsets of E_G and the $= 1$ case of Conjecture (2). Two distinct edges $e, f \in E_G$ are said to be $= 1$-*related* if they form a P_3 in G and, for every feasible solution x of $\max \mathcal{LP}(G)$, the corresponding edge variables satisfy

$$x_e + x_f = 1.$$

Definition 4.2.1 (Consistent Set)**.** *Let $E' \subseteq E_G$ be a maximal subset with $|E'| \geq 3$ such that, for every $e \in E'$, there exists $f \in E'$ such that e and f satisfy the $= 1$ relationship. We call E' a* ***consistent set*** *if there exists a partition*

$$E' = A \cup B, \quad A \cap B = \varnothing,$$

into nonempty sets A and B such that, for all $e, e' \in A$, the edges e and e' do not form a P_3 in G, and for all $f, f' \in B$, the edges f and f' do not form a P_3 in G.

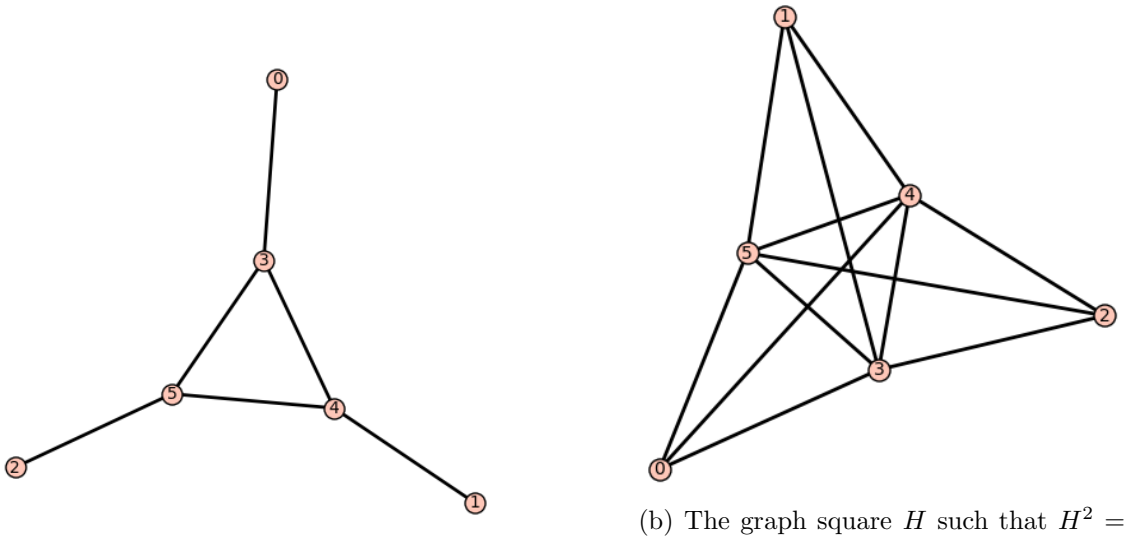
Consider Figure 4.2 where assigning $\alpha = x_{03}$ gives using Conjecture (2) that $x_{31} = 1 - \alpha$ since $g_{01} = 0$. Likewise as $g_{12} = 0$ have $x_{32} = \alpha$ and $x_{03} + x_{32} \leq 1$ such that $\alpha \leq \frac{1}{2}$. By symmetry this likewise holds for $\beta = x_{05} = x_{15}$, $1 - \beta = x_{52}$ and $\gamma = x_{42} = x_{41}$, $1 - \gamma = x_{40}$ giving $\beta, \gamma \leq \frac{1}{2}$.

Maximizing the objective function thus gives $\alpha = \beta = \gamma = \frac{1}{2}$ which is a fractional solution. To obtain an integral solution, one must instead select which edges within each inconsistent set are assigned value 1. For example, setting $\alpha = \beta = \gamma = 0$ with $x_{31} = x_{52} = x_{40} = 1$ and the remaining to 0 gives the solution Figure 4.2a, yet this is not doable in general. In this example, all of $\{(0,3),(3,1),(3,2)\}$, $\{(0,5),(1,5),(5,2)\}$ and $\{(4,2),(4,1),(4,0)\}$ are inconsistent sets.

Definition 4.2.2 (Inconsistent Set)**.** *Let $E' \subseteq E_G$ be a maximal subset with $|E'| \geq 3$ such that, for every $e \in E'$, there exists $f \in E'$ such that e and f satisfy the $= 1$ relationship. We call E' an* ***inconsistent set*** *if there is no partition*

$$E' = A \cup B, \quad A \cap B = \varnothing,$$

into nonempty sets A and B such that, for all $e, e' \in A$, the edges e and e' do not form a P_3 in G, and, for all $f, f' \in B$, the edges f and f' do not form a P_3 in G.



(a) The root graph G.

(b) The graph square H such that $H^2 = G$.

Figure 4.2: A six-vertex graph and its square. G only has one root, although there are different relabellings.

Conjecture 3. *Let G be a square graph whose consistent sets have a union forming a subgraph of G. Then the symbols associated with each consistent set can be fixed to values in $\{0, 1\}$ whilst preserving feasibility of the LP.*

Conjecture 4. *Let G be a square graph with at least one inconsistent set. Then at least one inconsistent set contains a star.*

Using Conjectures 3–4, we can formulate an edge enforcement algorithm that first identifies independent consistent sets and fixes their edges by setting the symbol to be 0 or 1. Before then identifying stars within the remaining inconsistent sets and fixing an edge within them.

Suppose that the edge enforcement procedure produces k consistent sets. An exhaustive procedure may need to consider up to 2^k different assignments and since k may depend on n, this is not polynomial in general. In particular, to obtain a polynomial bound from exhaustive enumeration, it would be necessary to prove that $k = O(\log n)$, since then

$$2^k = 2^{O(\log n)} = n^{O(1)}.$$

Hence, instead of exploring the full set, the greedy algorithm explores at most two LP feasibility tests for the selected group before moving to the next group and therefore avoids the exponential branching over combinations of consistent sets.

This gives the results in Table 4.4.

4.3 Duality

One of the strengths of a linear program in solving problems is the ability to leverage duality. In particular, the Karush-Kuhn-Tucker (KKT) conditions give deeper insight into the geometric interpretation of the feasible region through the introduction of dual variables. If the dual variable is non-zero, then the corresponding primal constraint is tight, whilst if the dual variable is zero,

Algorithm 3 Greedy edge enforcement

Input: A graph G
Output: A LP model (m, x)
Construct and solve $\mathcal{LP}(G)$.
Identify the consistent sets and inconsistent sets.
Mark all sets as unfixed.
while the current LP solution is fractional **do**
    if there is an unfixed consistent set **then**
        Select a consistent set C.
        Let a be the symbol associated with C.
        Add the constraint $a = 0$ and solve the LP.
        if the resulting LP is feasible **then**
            Keep the constraint $a = 0$.
        else
            Remove the constraint $a = 0$.
            Add the constraint $a = 1$ and solve the LP.
            if the resulting LP is feasible **then**
                Keep the constraint $a = 1$.
            else
                Remove the constraint $a = 1$.
            end if
        end if
        Mark C as fixed.
    else if there is an unfixed inconsistent set **then**
        Select an inconsistent set and identify a star
        with centre v.
        Order the edges of the star according to their
        current LP values, considering the most integral edge first.
        for each edge e of the selected star **do**
            Add the constraint $x_e = 1$ and solve the LP.
            if the resulting LP is feasible **then**
                Keep the constraint $x_e = 1$.
                Mark the inconsistent set as fixed.
                break.
            else
                Remove the constraint $x_e = 1$.
            end if
        end for
    else
        break.
    end if
end while
return the resulting LP model.

n	H exists		H does not exist	
	LP integral solution	LP fractional solution	LP infeasible	LP fractional solution
3	3	0	1	0
4	6	0	5	0
5	10	1	20	3
6	26	2	107	21
7	67	10	767	200
8	240	67	9380	2659

Table 4.4: Outcomes of the greedy iterative edge enforcement linear program using the max linear program as the base for graphs of different orders of n.

then the primal constraint is not satisfied. This enables solutions to the linear program to be viewed directly in the geometric region.

Importantly, the relationship between the primal and dual is highlighted by their solution to the objective function. For a linear program, these conditions reduce to primal feasibility, dual feasibility, and complementary slackness. In particular, when both the primal and dual are feasible, strong duality implies that their optimal objective values are equal.

Consider the dual formulation of the max objective linear program defined in Section 3.3 by its canonical form. Its dual can be rewritten as

Objective Function:

$$\max \quad \sum_{i=1}^{n}\sum_{j=1}^{n}\Big(-Mg_{ij}\gamma_{ij} + g_{ij}\delta_{ij} - \lambda_{ij}\Big) - \sum_{i=1}^{n}\sum_{k=1}^{n}\sum_{j=1}^{n}\big(\eta_{ikj} + \mu_{ikj}\big). \tag{4.1}$$

Dual Variables:

$$\alpha_i \in \mathbb{R},\ \forall i \in \{1,\dots,n\}, \quad \text{(associated with (2.12))}, \tag{4.2}$$
$$\beta_{ij} \in \mathbb{R},\ \forall i \neq j \in \{1,\dots,n\}, \text{(associated with (2.13))}, \tag{4.3}$$
$$\gamma_{ij} \geq 0,\ \forall i,j \in \{1,\dots,n\}, \quad \text{(associated with (2.14))}, \tag{4.4}$$
$$\delta_{ij} \geq 0,\ \forall i,j \in \{1,\dots,n\}, \quad \text{(associated with (2.15))}, \tag{4.5}$$
$$\epsilon_{ikj} \geq 0, \forall i,k,j \in \{1,\dots,n\}, \text{(associated with (2.16))}, \tag{4.6}$$
$$\zeta_{ikj} \geq 0, \forall i,k,j \in \{1,\dots,n\}, \text{(associated with (2.17))}, \tag{4.7}$$
$$\eta_{ikj} \geq 0, \forall i,k,j \in \{1,\dots,n\}, \text{(associated with (2.18))}, \tag{4.8}$$
$$\lambda_{ij} \geq 0,\ \forall i,j \in \{1,\dots,n\}, \quad \text{(associated with (2.19))}, \tag{4.9}$$
$$\mu_{ikj} \geq 0, \forall i,k,j \in \{1,\dots,n\}, \text{(associated with (2.20))}, \tag{4.10}$$
$$\theta_{ij} \in \mathbb{R},\ \forall i,j \in \{1,\dots,n\}, \quad \text{(associated with (2.21))}, \tag{4.11}$$
$$\iota_i \in \mathbb{R}, \quad \forall i \in \{1,\dots,n\}, \quad \text{(associated with (2.22))}. \tag{4.12}$$

Dual Constraints:

$\forall i, j \in \{1, \ldots, n\}$, each primal x_{ij} :

$$\begin{cases} -1 - \sum_{k=1}^{n} \epsilon_{iik} - \sum_{k=1}^{n} \zeta_{kii} + \sum_{k=1}^{n} \eta_{iik} + \sum_{k=1}^{n} \eta_{kii} + \iota_i + \lambda_{ii} \geq 0, & \text{if } j = i, \\ -1 + \alpha_i - \beta_{ij} - \sum_{k=1}^{n} \epsilon_{ikj} - \sum_{k=1}^{n} \zeta_{kij} + \sum_{k=1}^{n} \eta_{ikj} + \sum_{k=1}^{n} \eta_{kij} + \theta_{ij} - \theta_{ji} + \lambda_{ij} \geq 0, & \text{if } j \neq i, \end{cases} \tag{4.13}$$

$\forall i, k, j \in \{1, \ldots, n\}$, each primal y_{ikj} :

$$\begin{cases} -\alpha_i + \epsilon_{iki} + \zeta_{iki} - \eta_{iki} + \mu_{iki} \geq 0, & \text{if } j = i, \\ -\beta_{ij} + \epsilon_{ikj} + \zeta_{ikj} - \eta_{ikj} + \mu_{ikj} \geq 0, & \text{if } j \neq i, \end{cases} \tag{4.14}$$

$\forall i, j \in \{1, \ldots, n\}$, each primal z_{ij} :

$$\begin{cases} \alpha_i + \gamma_{ii} - \delta_{ii} = 0, & \text{if } j = i, \\ \beta_{ij} + \gamma_{ij} - \delta_{ij} = 0, & \text{if } j \neq i. \end{cases} \tag{4.15}$$

whereby the Lagrangian is

$$\begin{aligned} \mathcal{L}(x, y, z) = & -\sum_{i=1}^{n}\sum_{j=1}^{n} x_{ij} + \sum_{i=1}^{n} \alpha_i \Big(z_{ii} - \sum_{k=1}^{n} y_{iki} + \sum_{k=1}^{n} x_{ik} - x_{ii} \Big) + \sum_{i \neq j} \beta_{ij} \Big(z_{ij} - \sum_{k=1}^{n} y_{ikj} - x_{ij} \Big) \\ & + \sum_{i=1}^{n}\sum_{j=1}^{n} \gamma_{ij}(z_{ij} - M g_{ij}) + \sum_{i=1}^{n}\sum_{j=1}^{n} \delta_{ij}(-z_{ij} + g_{ij}) + \sum_{i=1}^{n}\sum_{k=1}^{n}\sum_{j=1}^{n} \epsilon_{ikj}(y_{ikj} - x_{ik}) \\ & + \sum_{i=1}^{n}\sum_{k=1}^{n}\sum_{j=1}^{n} \zeta_{ikj}(y_{ikj} - x_{kj}) + \sum_{i=1}^{n}\sum_{k=1}^{n}\sum_{j=1}^{n} \eta_{ikj}(x_{ik} + x_{kj} - y_{ikj} - 1) \\ & + \sum_{i=1}^{n}\sum_{j=1}^{n} \theta_{ij}(x_{ij} - x_{ji}) + \sum_{i=1}^{n} \iota_i x_{ii} + \sum_{i=1}^{n}\sum_{j=1}^{n} \lambda_{ij}(x_{ij} - 1) + \sum_{i=1}^{n}\sum_{k=1}^{n}\sum_{j=1}^{n} \mu_{ikj}(y_{ikj} - 1). \end{aligned}$$

Collecting terms gives

$$\begin{aligned} \mathcal{L}(x, y, z) = & \sum_{i=1}^{n}\sum_{j=1}^{n} x_{ij} \Bigg[-1 + \mathbf{1}_{\{j=i\}} \Big(\alpha_i - \sum_{k=1}^{n} \epsilon_{iki} - \sum_{k=1}^{n} \zeta_{kii} + \sum_{k=1}^{n} \eta_{iki} + \sum_{k=1}^{n} \eta_{kii} + \iota_i + \lambda_{ii} \Big) \\ & + \mathbf{1}_{\{j \neq i\}} \Big(\alpha_i - \beta_{ij} - \sum_{k=1}^{n} \epsilon_{ikj} - \sum_{k=1}^{n} \zeta_{kij} + \sum_{k=1}^{n} \eta_{ikj} + \sum_{k=1}^{n} \eta_{kij} + \lambda_{ij} \Big) + \theta_{ij} - \theta_{ji} \Bigg] \\ & + \sum_{i=1}^{n}\sum_{k=1}^{n}\sum_{j=1}^{n} y_{ikj} \Big[-\mathbf{1}_{\{j=i\}} \alpha_i - \mathbf{1}_{\{j \neq i\}} \beta_{ij} + \epsilon_{ikj} + \zeta_{ikj} - \eta_{ikj} + \mu_{ikj} \Big] \\ & + \sum_{i=1}^{n}\sum_{j=1}^{n} z_{ij} \Big[\mathbf{1}_{\{j=i\}} \alpha_i + \mathbf{1}_{\{j \neq i\}} \beta_{ij} + \gamma_{ij} - \delta_{ij} \Big] \\ & - \sum_{i=1}^{n}\sum_{j=1}^{n} M g_{ij} \gamma_{ij} + \sum_{i=1}^{n}\sum_{j=1}^{n} g_{ij} \delta_{ij} - \sum_{i=1}^{n}\sum_{k=1}^{n}\sum_{j=1}^{n} \eta_{ikj} - \sum_{i=1}^{n}\sum_{j=1}^{n} \lambda_{ij} - \sum_{i=1}^{n}\sum_{k=1}^{n}\sum_{j=1}^{n} \mu_{ikj}. \end{aligned}$$

In this scenario, the dual is tight and gives exactly the same objective function value as $\max \mathcal{LP}(G)$, consistent with the strong duality theorem. The results in Table 4.5 show that every

primal-feasible instance has a feasible dual, including both integral and fractional primal solutions, while the instances for which the primal is infeasible also have an infeasible dual. The KKT conditions, through complementary slackness, further characterize these optimal solutions by identifying which primal and dual constraints are tight. In particular, positive dual variables correspond to tight primal constraints, providing a potential means of identifying the constraints that determine the optimal objective value and of investigating the structure of fractional solutions.

n	H	Primal	Dual	Count
5	Exists	Integer	Feasible	9
5	Exists	Fractional	Feasible	2
5	Does not exist	Fractional	Feasible	3
5	Does not exist	Infeasible	Infeasible	20
6	Exists	Integer	Feasible	18
6	Exists	Fractional	Feasible	10
6	Does not exist	Fractional	Feasible	21
6	Does not exist	Infeasible	Infeasible	107
7	Exists	Integer	Feasible	37
7	Exists	Fractional	Feasible	40
7	Does not exist	Fractional	Feasible	200
7	Does not exist	Infeasible	Infeasible	767

Table 4.5: Primal and dual LP outcomes for graphs of different orders of n.

It remains to explore this more by investigating the structure of the optimal dual solutions, the relationship between positive dual variables and fractional primal variables, and whether dual information can be used to derive additional valid constraints for the square-root recognition problem.

4.4 Rounding

Finally, we consider a simple heuristic for further refining fractional LP solutions - rounding. Since each edge variable takes a value in $[0, 1]$, many strategies are possible, ranging from probabilistic rounding to systematically fixing each fractional edge in both possible ways. Because every edge variable must ultimately take a value in $\{0, 1\}$ in an integral solution, a more exhaustive approach is to branch on each fractional variable, assigning it one of its two possible integral values. This produces a binary search tree whose depth is bounded by the number of fractional variables.

If the number of fractional variables is bounded by $O(\log n)$, where $n = |V_G|$, then the search tree has at most $2^{O(\log n)} = n^{O(1)}$ leaves. Therefore, provided that the work performed at each node of the search tree is polynomial in n, the overall algorithm runs in polynomial time. Classical techniques such as pruning, bounding, and depth-first or breadth-first search may then be leveraged to evaluate all end nodes efficiently: pruning discards branches whose LP bound exceeds the incumbent, bounding limits the depth of exploration, and the search order determines how quickly an integral solution is found.

In Algorithm 4, we use only a simple greedy rounding procedure. At each iteration, the current fractional LP solution is examined and a single fractional edge is selected. The selection is based on the distance of its value from 0.5, so that fractional values closer to 0 or 1 are considered first. The selected edge is then fixed to 0 if its value is below 0.5 and to 1 otherwise. The LP is subsequently resolved with all previous fixings retained.

If the resulting LP is infeasible, the rounding procedure terminates and the previous feasible fractional solution is retained. If the resulting LP remains fractional, another fractional edge is selected and the process continues. The procedure terminates when an integral solution is obtained or when no further feasible fixing can be made.

Algorithm 4 Greedy LP Rounding

Input: A graph G
Output: A LP model (m, x)
Initialize $F = \emptyset$ and construct $\mathcal{UVLP}(G)$.
Solve $\mathcal{UVLP}(G)$.
while the current LP solution is fractional **do**
    For each fractional edge ab, compute its rounding score $s_{ab} = |x_{ab} - 0.5|$.
    Select a fractional edge ab with maximum score.
    Set

$$\widehat{x}_{ab} = \widehat{x}_{ba} = \begin{cases} 0, & x_{ab} < 0.5, \\ 1, & x_{ab} \geq 0.5. \end{cases}$$

    Add ab with its assigned value to F.
    Reconstruct $\mathcal{UVLP}(G)$ with all fixings in F.
    Solve $\mathcal{UVLP}(G) + F$.
    if $\mathcal{UVLP}(G) + F$ is infeasible **then**
        return the previous feasible solution (m, x).
    end if
    if $\mathcal{UVLP}(G) + F$ is integral **then**
        return the integral solution (m, x).
    end if
end while
return (m, x).

The procedure is greedy because only one fixing is selected at each iteration and no alternative fixing is explored. Thus, unlike exhaustive rounding or branch-and-bound approaches, it does not construct a binary search tree of possible fixings. Since there are at most $\binom{|V_G|}{2}$ edge variables to fix and each iteration requires one LP solve, the rounding procedure requires at most $O(n^2)$ LP solves. Consequently, as $\mathcal{UVLP}(G)$ can be solved in polynomial time, the greedy rounding procedure is polynomial-time.

The choice of the base linear program is important for the effectiveness of the subsequent rounding procedure. The rounding procedure begins with a fractional solution and iteratively fixes selected edge variables. Consequently, the constraints imposed by the base formulation determine which fractional solutions can be successfully rounded and which become infeasible during the fixing process.

The outcomes obtained by using the maximum-objective linear program and the unicliqual vertex linear program as base formulations are shown in Table 4.6 and Table 4.7. These results demonstrate that the choice of base formulation has a substantial effect on the effectiveness of the subsequent rounding procedure. Although $\mathcal{UVLP}$ produces a smaller increase in successfully rounded solutions, the improvement remains significant, as the additional structural constraints enable more root instances to be correctly classified.

n	H exists		H does not exist		Integral ← Fractional
	LP integral	LP fractional	LP infeasible	LP fractional	
3	3	0	1	0	0
4	6	0	5	0	0
5	11	0	20	3	2
6	27	1	107	21	9
7	73	4	767	200	35
8	245	62	9380	2659	151

Table 4.6: Outcomes of the greedy rounding linear program using the max linear program as the base for graphs of different orders of n. The final column gives the number of initially fractional instances that are converted to integral solutions by the rounding procedure.

n	H exists		H does not exist		Integral ← Fractional
	LP integral	LP fractional	LP infeasible	LP fractional	
3	3	0	1	0	0
4	6	0	5	0	0
5	11	0	21	2	1
6	28	0	117	11	6
7	76	1	852	115	25
8	268	39	10271	1768	120

Table 4.7: Outcomes of the greedy rounding linear program using the unicliqual vertex linear program as the base for graphs of different orders of n. The final column gives the number of initially fractional instances that are converted to integral solutions by the rounding procedure.

Bibliography

[BK73] Coen Bron and Joep Kerbosch. "Algorithm 457: Finding all cliques of an undirected graph". In: *Communications of the ACM* 16.9 (1973), pp. 575–577.

[BMR14] Stephan Brandt, Janina Muttel, and Dieter Rautenbach. "The circumference of the square of a connected graph". In: *Combinatorica* 34.5 (2014), pp. 547–559.

[Coc+13] Manfred Cochefert, Jean-François Couturier, Petr A. Golovach, Dieter Kratsch, and Daniël Paulusma. "Sparse Square Roots". In: Lecture Notes in Computer Science 8165 (2013), pp. 177–188.

[Coo71] Stephen A. Cook. "The Complexity of Theorem-Proving Procedures". In: *Proceedings of the Third Annual ACM Symposium on Theory of Computing* (1971), pp. 151–158.

[Duf25] Christopher Duffy. "Quasi-Transitive Mixed Graphs and Undirected Squares of Oriented Graphs". In: *Australasian Journal of Combinatorics* 92.2 (2025), pp. 142–163.

[Far+09] Babak Farzad, Lap Chi Lau, Van Bang Le, and Nguyen Ngoc Tuy. "Computing Graph Roots Without Short Cycles". In: *Proceedings of the 26th International Symposium on Theoretical Aspects of Computer Science (STACS 2009)*. Vol. 3. Leibniz International Proceedings in Informatics (LIPIcs). 2009, pp. 397–408.

[FBB23] Leon Festinger, Françoise Bahoken, and Laurent Beauguitte. "Leon Festinger, 1949, The Analysis of Sociograms Using Matrix Algebra. Version bilingue et commentée". In: (2023).

[FK12] Babak Farzad and Majid Karimi. "Square-Root Finding Problem In Graphs, A Complete Dichotomy Theorem". In: (2012).

[Fle74] Herbert Fleischner. "The square of every nonseparable graph is Hamiltonian". In: *Journal of Combinatorial Theory, Series B* 16.1 (1974), pp. 29–34.

[Gel70] Dennis P. Geller. "The Square Root of a Digraph". In: *Journal of Combinatorial Theory* 9.1 (1970), pp. 65–68.

[Gon91] Clovis C. Gonzaga. "Large Step Path-Following Methods for Linear Programming, Part I: Barrier Function Method". In: *SIAM Journal on Optimization* 1.2 (1991), pp. 268–279.

[Gur26] Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual*. 2026. URL: `https://www.gurobi.com`.

[HKT67] Frank Harary, Richard M. Karp, and William T. Tutte. "A criterion for planarity of the square of a graph". In: *Journal of Combinatorial Theory* 2 (1967), pp. 406–410.

[Kar72] Richard M. Karp. "Reducibility Among Combinatorial Problems". In: *Complexity of Computer Computations* (1972), pp. 85–103.

[Kön36] Dénes König. *Theorie der endlichen und unendlichen Graphen*. Akademische Verlagsgesellschaft, 1936.

[Lau06] Lap Chi Lau. "Bipartite Roots of Graphs". In: *ACM Transactions on Algorithms* 2.2 (2006), pp. 178–208.

[LC04] Lap Chi Lau and Derek G. Corneil. "Recognizing Powers of Proper Interval, Split, and Chordal Graphs". In: *SIAM Journal on Discrete Mathematics* 18.1 (2004), pp. 83–102.

[LS95] Yaw-Ling Lin and Steven S. Skiena. "Algorithms for square roots of graphs". In: *SIAM Journal on Discrete Mathematics* 8.1 (1995), pp. 99–118.

[LT07] Van Bang Le and Nguyen Ngoc Tuy. "The square of a block graph". In: *Discrete Mathematics* 307.21 (2007), pp. 2706–2715.

[MS94] Rajeev Motwani and Madhu Sudan. "Computing roots of graphs is hard". In: *Discrete Applied Mathematics* 54.1 (1994), pp. 81–88.

[Muk67] A. Mukhopadhyay. "The Square Root of a Graph". In: *Journal of Combinatorial Theory* 2 (1967), pp. 290–295.

[PA23] Or Peres and C. Avin. "Distributed Demand-aware Network Design using Bounded Square Root of Graphs". In: *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications* (2023), pp. 1–10.

[RH60] Ian C. Ross and Frank Harary. "The square of a tree". In: *The Bell System Technical Journal* 39.3 (1960), pp. 641–647.

[Sch78] Thomas J. Schaefer. "The complexity of satisfiability problems". In: *Proceedings of the Tenth Annual ACM Symposium on Theory of Computing* (1978), pp. 216–226.

[The26] The Sage Developers. *SageMath, the Sage Mathematics Software System (Version 10.7)*. `https://www.sagemath.org`. 2026.

[Tse68] Grigori S. Tseitin. "On the Complexity of Derivation in Propositional Calculus". In: *Studies in Constructive Mathematics and Mathematical Logic, Part II* (1968), pp. 115–125.

[Zho+25] Bo Zhou, Siyang Liu, Lei Chen, and Qi Dai. "Graph-Root: Prediction of Root-Associated Proteins in Maize, Sorghum, And Soybean Based on Graph Convolutional Network and Network Embedding Method". In: *Current Bioinformatics* (2025), pp. 828–839.

Declaration

I acknowledge the use of ChatGPT (OpenAI) in the preparation of this thesis.

ChatGPT was used to assist with the formulation and refinement of diagrams and the testing of the code framework used for computational experiments.

A full record of prompts and outputs is available upon request.

Listing 1: Integer Program.

```
def integer_program(G):
    m = MixedIntegerLinearProgram(solver='Gurobi', maximization=False)
    m.solver_parameter("Method", 2)

    x = m.new_variable(integer=True, nonnegative=True)
    y = m.new_variable(integer=True, nonnegative=True)
    z = m.new_variable(integer=True, nonnegative=True)

    A_G = G.adjacency_matrix()
    n = A_G.nrows()
    M = n**2

    # Indicator argument
    for i in range(n):
        for j in range(n):
            if i == j:
                m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
                    - sum(x[i,k] for k in range(n)) + x[i,j]) == 0)
            else:
                m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
                    + x[i,j]) == 0)

    # Indicator function
    for i in range(n):
        for j in range(n):
            m.add_constraint(z[i,j] - M * A_G[i][j] <= 0)
            m.add_constraint(-z[i,j] + A_G[i][j] <= 0)

    # Product
    for i in range(n):
        for j in range(n):
            for k in range(n):
                m.add_constraint(y[i,k,j] - x[i,k] <= 0)
                m.add_constraint(y[i,k,j] - x[k,j] <= 0)
                m.add_constraint(- y[i,k,j] + x[i,k] + x[k,j] - 1 <= 0)

    # Symmetry
    for i in range(n):
        for j in range(n):
            m.add_constraint(x[i,j] == x[j,i])

    # Diagonal
    for i in range(n):
        m.add_constraint(x[i,i] == 0)

```

```
    # Variables
    for i in range(n):
        for j in range(n):
            m.add_constraint(x[i,j] - 1 <= 0)
            for k in range(n):
                m.add_constraint(y[i,k,j] - 1 <= 0)

    return m, x
```

Listing 2: Linear Program.

```
def linear_program(G):
    m = MixedIntegerLinearProgram(solver='Gurobi', maximization=False)
    m.solver_parameter("Method", 2)

    x = m.new_variable(real=True, nonnegative=True)
    y = m.new_variable(real=True, nonnegative=True)
    z = m.new_variable(real=True, nonnegative=True)

    A_G = G.adjacency_matrix()
    n = A_G.nrows()
    M = n**2

    # Indicator argument
    for i in range(n):
        for j in range(n):
            if i == j:
                m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
                    - sum(x[i,k] for k in range(n)) + x[i,j]) == 0)
            else:
                m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
                    + x[i,j]) == 0)

    # Indicator function
    for i in range(n):
        for j in range(n):
            m.add_constraint(z[i,j] - M * A_G[i][j] <= 0)
            m.add_constraint(-z[i,j] + A_G[i][j] <= 0)

    # Product
    for i in range(n):
        for j in range(n):
            for k in range(n):
                m.add_constraint(y[i,k,j] - x[i,k] <= 0)
                m.add_constraint(y[i,k,j] - x[k,j] <= 0)
                m.add_constraint(- y[i,k,j] + x[i,k] + x[k,j] - 1 <= 0)

    # Symmetry
    for i in range(n):
        for j in range(n):
            m.add_constraint(x[i,j] == x[j,i])

    # Diagonal
    for i in range(n):
```

```
        m.add_constraint(x[i,i] == 0)

    # Variables
    for i in range(n):
        for j in range(n):
            m.add_constraint(x[i,j] - 1 <= 0)
            for k in range(n):
                m.add_constraint(y[i,k,j] - 1 <= 0)
    return m, x
```

Listing 3: Max Linear Program.

```
def max_linear_program(G):
    m = MixedIntegerLinearProgram(solver='Gurobi', maximization=False)
    m.solver_parameter("Method", 2)

    x = m.new_variable(real=True, nonnegative=True)
    y = m.new_variable(real=True, nonnegative=True)
    z = m.new_variable(real=True, nonnegative=True)

    A_G = G.adjacency_matrix()
    n = A_G.nrows()
    M = n**2

    # Indicator argument
    for i in range(n):
        for j in range(n):
            if i == j:
                m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
                    - sum(x[i,k] for k in range(n)) + x[i,j]) == 0)
            else:
                m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
                    + x[i,j]) == 0)

    # Indicator function
    for i in range(n):
        for j in range(n):
            m.add_constraint(z[i,j] - M * A_G[i][j] <= 0)
            m.add_constraint(-z[i,j] + A_G[i][j] <= 0)

    # Product
    for i in range(n):
        for j in range(n):
            for k in range(n):
                m.add_constraint(y[i,k,j] - x[i,k] <= 0)
                m.add_constraint(y[i,k,j] - x[k,j] <= 0)
                m.add_constraint(- y[i,k,j] + x[i,k] + x[k,j] - 1 <= 0)

    # Symmetry
    for i in range(n):
        for j in range(n):
            m.add_constraint(x[i,j] == x[j,i])

    # Diagonal
```

```
    for i in range(n):
        m.add_constraint(x[i,i] == 0)

    # Variables
    for i in range(n):
        for j in range(n):
            m.add_constraint(x[i,j] - 1 <= 0)
            for k in range(n):
                m.add_constraint(y[i,k,j] - 1 <= 0)

    m.set_objective(- m.sum(x[i,j] for i in range(n) for j in range(n)))
    return m, x
```

Listing 4: Min Linear Program.

```
def min_linear_program(G):
    m = MixedIntegerLinearProgram(solver='Gurobi', maximization=False)
    m.solver_parameter("Method", 2)

    x = m.new_variable(real=True, nonnegative=True)
    y = m.new_variable(real=True, nonnegative=True)
    z = m.new_variable(real=True, nonnegative=True)

    A_G = G.adjacency_matrix()
    n = A_G.nrows()
    M = n**2

    # Indicator argument
    for i in range(n):
        for j in range(n):
            if i == j:
                m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
                    - sum(x[i,k] for k in range(n)) + x[i,j]) == 0)
            else:
                m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
                    + x[i,j]) == 0)

    # Indicator function
    for i in range(n):
        for j in range(n):
            m.add_constraint(z[i,j] - M * A_G[i][j] <= 0)
            m.add_constraint(-z[i,j] + A_G[i][j] <= 0)

    # Product
    for i in range(n):
        for j in range(n):
            for k in range(n):
                m.add_constraint(y[i,k,j] - x[i,k] <= 0)
                m.add_constraint(y[i,k,j] - x[k,j] <= 0)
                m.add_constraint(- y[i,k,j] + x[i,k] + x[k,j] - 1 <= 0)

    # Symmetry
    for i in range(n):
        for j in range(n):
```

```
            m.add_constraint(x[i,j] == x[j,i])

    # Diagonal
    for i in range(n):
        m.add_constraint(x[i,i] == 0)

    # Variables
    for i in range(n):
        for j in range(n):
            m.add_constraint(x[i,j] - 1 <= 0)
            for k in range(n):
                m.add_constraint(y[i,k,j] - 1 <= 0)

    m.set_objective(m.sum(x[i,j] for i in range(n) for j in range(n)))
    return m, x
```

Listing 5: Unicliqual Vertex Linear Program.

```
def find_leaf_stem(G):
    n = G.order()
    N = {v: set(G.neighbors(v)) | {v} for v in range(n)}
    signature = {v: frozenset(N[v]) for v in range(n)}

    contained = [[False] * n for _ in range(n)]
    proper = [[False] * n for _ in range(n)]

    for u in range(n):
        for v in range(n):
            if N[u] <= N[v]:
                contained[u][v] = True
                proper[u][v] = N[u] != N[v]

    for s in range(n):
        for u in N[s] - {s}:
            if not proper[s][u]:
                continue
            for v in N[u] - {u}:
                if not contained[u][v]:
                    continue
                leaves = N[s] - {u, v}
                if all(signature[w] == signature[s] for w in leaves):
                    return {"leaf": s, "stem": u, "vertex": v, "leaves":
                        leaves}

    return None

def unicliqual_vertex_linear_program(G):
    m = MixedIntegerLinearProgram(solver='Gurobi', maximization=False)
    m.solver_parameter("Method", 2)

    x = m.new_variable(real=True, nonnegative=True)
    y = m.new_variable(real=True, nonnegative=True)
    z = m.new_variable(real=True, nonnegative=True)

```

```
36      A_G = G.adjacency_matrix()
37      n = A_G.nrows()
38      M = n**2
39
40      # Indicator argument
41      for i in range(n):
42          for j in range(n):
43              if i == j:
44                  m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
                        - sum(x[i,k] for k in range(n)) + x[i,j]) == 0)
45              else:
46                  m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
                        + x[i,j]) == 0)
47
48      # Indicator function
49      for i in range(n):
50          for j in range(n):
51              m.add_constraint(z[i,j] - M * A_G[i][j] <= 0)
52              m.add_constraint(-z[i,j] + A_G[i][j] <= 0)
53
54      # Product
55      for i in range(n):
56          for j in range(n):
57              for k in range(n):
58                  m.add_constraint(y[i,k,j] - x[i,k] <= 0)
59                  m.add_constraint(y[i,k,j] - x[k,j] <= 0)
60                  m.add_constraint(- y[i,k,j] + x[i,k] + x[k,j] - 1 <= 0)
61
62      # Symmetry
63      for i in range(n):
64          for j in range(n):
65              m.add_constraint(x[i,j] == x[j,i])
66
67      # Diagonal
68      for i in range(n):
69          m.add_constraint(x[i,i] == 0)
70
71      # Variables
72      for i in range(n):
73          for j in range(n):
74              m.add_constraint(x[i,j] - 1 <= 0)
75              for k in range(n):
76                  m.add_constraint(y[i,k,j] - 1 <= 0)
77
78      m.set_objective(-m .sum(x[i, j] for i in range(n) for j in range(n)))
79
80      # Find a leaf-stem structure using the largest closed neighbourhood
            clique
81      structure = find_leaf_stem(G)
82
83      if structure is None:
84
85          return m, x
86
```

```
    s = structure["leaf"]
    u = structure["stem"]

    m.add_constraint(x[s, u] == 1)
    m.add_constraint(x[u, s] == 1)

    return m, x
```

Listing 6: Vertex Deletion Linear Program.

```
def vertex_deletion_linear_program(G):
    fixed_edges = set()
    m, x = unicliqual_vertex_linear_program(G)
    for u, v in fixed_edges:
        m.add_constraint(x[u, v] == 1)
        m.add_constraint(x[v, u] == 1)
    m.solve()

    n = G.order()
    solution_x = m.get_values(x)
    integral_edges = {tuple(sorted((i, j))) for i in range(n) for j in
       range(i + 1, n) if abs(solution_x[i, j] - 1) <= 1e-6}
    fixed_edges.update(integral_edges)
    is_integral = not any(abs(solution_x[i, j] - round(solution_x[i, j]))
       > 1e-6 for i in range(n) for j in range(n))
    if is_integral:
        return m, x

    unfixed_degree = {v: sum(tuple(sorted((u, v))) not in fixed_edges for
       u in G.neighbors(v)) for v in G}
    v = min(unfixed_degree, key=lambda u: unfixed_degree[u])
    remaining_vertices = [u for u in G if u != v]
    G_minus = G.subgraph(remaining_vertices)
    relabel_map = {u: i for i, u in enumerate(remaining_vertices)}
    G_minus.relabel(relabel_map)
    fixed_edges_minus = {tuple(sorted((relabel_map[u], relabel_map[w])))
       for u, w in fixed_edges if u in relabel_map and w in relabel_map}

    H_minus = None
    try:
        m_minus, x_minus = unicliqual_vertex_linear_program(G_minus)
        for u, w in fixed_edges_minus:
            m_minus.add_constraint(x_minus[u, w] == 1)
            m_minus.add_constraint(x_minus[w, u] == 1)
        m_minus.solve()
        n_minus = G_minus.order()
        solution_x_minus = m_minus.get_values(x_minus)
        is_integral_minus = not any(abs(solution_x_minus[i, j] - round(
           solution_x_minus[i, j])) > 1e-6 for i in range(n_minus) for j
           in range(n_minus))
        if is_integral_minus:
            H_minus = Graph(matrix([[round(solution_x_minus[i, j]) for j
               in range(n_minus)] for i in range(n_minus)]))
    except MIPSolverException:
```

```
        pass

    if H_minus is not None:
        m_test, x_test = unicliqual_vertex_linear_program(G)
        for u, w in fixed_edges:
            m_test.add_constraint(x_test[u, w] == 1)
            m_test.add_constraint(x_test[w, u] == 1)
        for i, a in enumerate(remaining_vertices):
            for j, b in enumerate(remaining_vertices):
                if a != b and H_minus.has_edge(i, j):
                    m_test.add_constraint(x_test[a, b] == 1)
        try:
            m_test.solve()
            solution_x_test = m_test.get_values(x_test)
            new_fixed_edges = {tuple(sorted((i, j))) for i in range(n) for
                j in range(i + 1, n) if abs(solution_x_test[i, j] - 1) <=
               1e-6}
            added_edges = new_fixed_edges - fixed_edges
            fixed_edges.update(new_fixed_edges)
            is_integral_test = not any(abs(solution_x_test[i, j] - round(
               solution_x_test[i, j])) > 1e-6 for i in range(n) for j in
               range(n))
            if is_integral_test:
                return m_test, x_test
            if added_edges:
                m, x, solution_x = m_test, x_test, solution_x_test
                return vertex_deletion_linear_program(G)
        except MIPSolverException:
            pass

    return m, x
```

Listing 7: Recursive Vertex Deletion Linear Program.

```
def vertex_deletion_recursive_linear_program(G):
    fixed_edges = set()
    m, x = unicliqual_vertex_linear_program(G)
    m.solve()

    n = G.order()
    solution_x = m.get_values(x)
    integral_edges = {tuple(sorted((i, j))) for i in range(n) for j in
       range(i + 1, n) if abs(solution_x[i, j] - 1) <= 1e-6}
    fixed_edges.update(integral_edges)
    is_integral = not any(abs(solution_x[i, j] - round(solution_x[i, j]))
       > 1e-6 for i in range(n) for j in range(n))
    if is_integral:
        return m, x

    unfixed_degree = {v: sum(tuple(sorted((u, v))) not in fixed_edges for
       u in G.neighbors(v)) for v in G}
    v = min(unfixed_degree, key=lambda u: unfixed_degree[u])
    remaining_vertices = [u for u in G if u != v]
    G_minus = G.subgraph(remaining_vertices)
```

```
    relabel_map = {u: i for i, u in enumerate(remaining_vertices)}
    G_minus.relabel(relabel_map)
    fixed_edges_minus = {tuple(sorted((relabel_map[u], relabel_map[w])))
       for u, w in fixed_edges if u in relabel_map and w in relabel_map}

    H_minus = None
    try:
        m_minus, x_minus = unicliqual_vertex_linear_program(G_minus)
        for u, w in fixed_edges_minus:
            m_minus.add_constraint(x_minus[u, w] == 1)
            m_minus.add_constraint(x_minus[w, u] == 1)
        m_minus.solve()
        n_minus = G_minus.order()
        solution_x_minus = m_minus.get_values(x_minus)
        is_integral_minus = not any(abs(solution_x_minus[i, j] - round(
           solution_x_minus[i, j])) > 1e-6 for i in range(n_minus) for j
           in range(n_minus))
        if is_integral_minus:
            H_minus = Graph(matrix([[round(solution_x_minus[i, j]) for j
               in range(n_minus)] for i in range(n_minus)]))
        else:
            m_recursive, x_recursive =
               vertex_deletion_recursive_linear_program(G_minus)
            solution_x_minus = m_recursive.get_values(x_recursive)
            is_integral_recursive = not any(abs(solution_x_minus[i, j] -
               round(solution_x_minus[i, j])) > 1e-6 for i in range(
               n_minus) for j in range(n_minus))
            if is_integral_recursive:
                H_minus = Graph(matrix([[round(solution_x_minus[i, j]) for
                    j in range(n_minus)] for i in range(n_minus)]))
    except MIPSolverException:
        pass

    if H_minus is not None:
        m_test, x_test = unicliqual_vertex_linear_program(G)
        for u, w in fixed_edges:
            m_test.add_constraint(x_test[u, w] == 1)
            m_test.add_constraint(x_test[w, u] == 1)
        for i, a in enumerate(remaining_vertices):
            for j, b in enumerate(remaining_vertices):
                if a != b and H_minus.has_edge(i, j):
                    m_test.add_constraint(x_test[a, b] == 1)
        try:
            m_test.solve()
            solution_x_test = m_test.get_values(x_test)
            new_fixed_edges = {tuple(sorted((i, j))) for i in range(n) for
                j in range(i + 1, n) if abs(solution_x_test[i, j] - 1) <=
               1e-6}
            added_edges = new_fixed_edges - fixed_edges
            fixed_edges.update(new_fixed_edges)
            is_integral_test = not any(abs(solution_x_test[i, j] - round(
               solution_x_test[i, j])) > 1e-6 for i in range(n) for j in
               range(n))
            if is_integral_test:
```

```
                return m_test, x_test
            if added_edges:
                m, x, solution_x = m_test, x_test, solution_x_test
        except MIPSolverException:
            pass

    return m, x
```

Listing 8: Max Edge Enforcment Linear Program.

```
def two_edge_triangles_from_edges(G):
    A = G.adjacency_matrix()
    n = G.num_verts()
    triangles = []
    for u in range(n):
        for v in range(u + 1, n):
            for w in range(v + 1, n):
                edges = A[u][v] + A[u][w] + A[v][w]
                if edges == 2:
                    triangles.append((u, v, w))
    return triangles

def present_edges(i, j, k, G):
    A = G.adjacency_matrix()
    edges = []
    if A[i][j]:
        edges.append((i, j))
    if A[i][k]:
        edges.append((i, k))
    if A[j][k]:
        edges.append((j, k))
    return edges

def is_fixed(value, tol=1e-8):
    return abs(value) < tol or abs(value - 1) < tol

def invalid_group(G, group_edges):
    group_edges = {tuple(sorted(e)) for e in group_edges}
    for v in G.vertices():
        incident_edges = {tuple(sorted(e)) for e in G.edges_incident(v,
            labels=False)}
        if not incident_edges:
            continue
        if incident_edges.issubset(group_edges):
            return True
    return False

def find_consistent_inconsistent_sets(G, solution=None):
    triangles = two_edge_triangles_from_edges(G)
    H = Graph()
    for tri in triangles:
        e1, e2 = present_edges(*tri, G)
        if solution is not None:
            if is_fixed(solution[e1]):
```

```
                continue
            if is_fixed(solution[e2]):
                continue
        H.add_vertex(e1)
        H.add_vertex(e2)
        H.add_edge(e1, e2)
    consistent_sets = []
    inconsistent_sets = []
    for component in H.connected_components():
        C = H.subgraph(component)
        if C.num_verts() < 3:
            continue
        group_edges = sorted(component)
        if C.is_bipartite():
            if invalid_group(G, group_edges):
                inconsistent_sets.append({
                    "edges": group_edges,
                })
                continue
            part0, part1 = C.bipartite_sets()
            consistent_sets.append({
                "edges": group_edges,
                "part0": sorted(part0),
                "part1": sorted(part1),
            })
        else:
            inconsistent_sets.append({
                "edges": group_edges,
            })
    return consistent_sets, inconsistent_sets

def edge_enforcement(G):
    consistent_sets, inconsistent_sets = find_consistent_inconsistent_sets
        (G)
    consistent_assignment = {}
    inconsistent_assignment = {}
    consistent_variables = []
    inconsistent_variables = []
    next_var = 0
    # Consistent sets
    for group in consistent_sets:
        a = var(f"a{next_var}")
        next_var += 1
        consistent_variables.append(a)
        for edge in group["part0"]:
            consistent_assignment[edge] = a
        for edge in group["part1"]:
            consistent_assignment[edge] = 1 - a
    # Inconsistent sets
    for group in inconsistent_sets:
        b = var(f"b{next_var}")
        next_var += 1
        inconsistent_variables.append(b)
        for edge in group["edges"]:
```

```
            inconsistent_assignment[edge] = b

    return consistent_assignment, inconsistent_assignment,
       consistent_variables, inconsistent_variables, consistent_sets,
       inconsistent_sets

def max_edge_enforcement_no_fixing_linear_program(G):
    m, x = max_linear_program(G)

    triangles = two_edge_triangles_from_edges(G)
    consistent_assignment, inconsistent_assignment, consistent_vars,
       inconsistent_vars, consistent_sets, inconsistent_sets =
       edge_enforcement(G)

    assignment = { **consistent_assignment, **inconsistent_assignment}
    variables = consistent_vars + inconsistent_vars

    var_index = {v: i for i, v in enumerate(variables)}

    group_lp = m.new_variable(real=True)

    for i in range(len(variables)):
        m.add_constraint(group_lp[i] >= 0)
        m.add_constraint(group_lp[i] <= 1)

    for group, a in zip(consistent_sets, consistent_vars):
        idx = var_index[a]
        for edge in group["part0"]:
            m.add_constraint(x[edge] == group_lp[idx])
        for edge in group["part1"]:
            m.add_constraint(x[edge] == 1 - group_lp[idx])

    return m, x, group_lp, variables, assignment


def choose_inconsistent_star(G, inconsistent_sets):
    if not inconsistent_sets:
        return None
    for group in inconsistent_sets:
        stars = defaultdict(list)
        for u, v in group["edges"]:
            edge = tuple(sorted((u, v)))
            stars[u].append(edge)
            stars[v].append(edge)
        if not stars:
            continue
        centre = max(stars, key=lambda v: (len(stars[v]), -v))
        return {
            "centre": centre,
            "edges": sorted(stars[centre])
        }
    return None

def choose_star_edge(star, solution_x):
```

```
147     edges = sorted(star["edges"])
148     return min(edges, key=lambda e: (min(solution_x[e], 1 - solution_x[e])
            , e))
149
150 def choose_consistent_group(consistent_sets):
151     if not consistent_sets:
152         return None
153     return sorted(consistent_sets, key=lambda g: sorted(g))[0]
154
155 def choose_group_symbol(group):
156     return min(group["edges"])
157
158
159 def max_edge_enforcement_fixing_linear_program(G):
160     m, x, a_lp, variables, assignment =
            max_edge_enforcement_no_fixing_linear_program(G)
161     n = G.order()
162
163     try:
164         m.solve()
165     except MIPSolverException:
166         return m, x
167
168     solution_x = m.get_values(x)
169     is_noninteger = False
170     A_H = matrix([[solution_x[i, j] for j in range(n)] for i in range(n)])
171     if any(abs(A_H[i, j] - round(A_H[i, j])) > 1e-6 for i in range(n) for
            j in range(n)):
172         is_noninteger = True
173
174     if not is_noninteger:
175         return m, x
176
177     consistent_assignment, inconsistent_assignment, consistent_vars,
            inconsistent_vars, consistent_sets, inconsistent_sets =
            edge_enforcement(G)
178     var_index = { v: i for i, v in enumerate(variables) }
179
180     if consistent_sets:
181         group = choose_consistent_group(consistent_sets)
182         group_index = consistent_sets.index(group)
183         symbol = consistent_vars[group_index]
184         idx = var_index[symbol]
185         m.add_constraint(a_lp[idx] == 0)
186         try:
187             m.solve()
188             return m, x
189         except MIPSolverException:
190             pass
191
192         m, x, a_lp, variables, assignment =
                max_edge_enforcement_no_fixing_linear_program(G)
193         m.add_constraint(a_lp[idx] == 1)
194         try:
```

```
            m.solve()
            return m, x
        except MIPSolverException:
            pass
    if inconsistent_sets:
        star = choose_inconsistent_star(G, inconsistent_sets)
        if star is None:
            return m, x

        for edge in sorted(star["edges"]):
            value = int(solution_x[edge] >= 0.5)

            m_branch, x_branch, a_lp_branch, variables_branch,
               assignment_branch =
               max_edge_enforcement_no_fixing_linear_program(G)
            m_branch.add_constraint(x_branch[edge] == value)
            try:
                m_branch.solve()
                return m_branch, x_branch

            except MIPSolverException:
                continue
    return m, x

def max_edge_enforcement_iterative_linear_program(G):
    m, x, a_lp, variables, assignment =
       max_edge_enforcement_no_fixing_linear_program(G)
    n = G.order()

    consistent_assignment, inconsistent_assignment, consistent_vars,
       inconsistent_vars, consistent_sets, inconsistent_sets =
       edge_enforcement(G)
    var_index = {v: i for i, v in enumerate(variables)}
    fixed_consistent_sets = set()
    fixed_inconsistent_sets = set()
    fixed_constraints = []

    while True:
        try:
            m.solve()
        except MIPSolverException:
            return m, x

        solution_x = m.get_values(x)
        is_noninteger = any(abs(solution_x[i, j] - round(solution_x[i, j])
           ) > 1e-6 for i in range(n) for j in range(n))

        if not is_noninteger:
            return m, x

        available_consistent_sets = [(i, group) for i, group in enumerate(
           consistent_sets) if i not in fixed_consistent_sets]
        available_inconsistent_sets = [(i, group) for i, group in
           enumerate(inconsistent_sets) if i not in
```

```
            fixed_inconsistent_sets]

        if available_consistent_sets:
            group_index, group = available_consistent_sets[0]
            symbol = consistent_vars[group_index]
            idx = var_index[symbol]

            m0, x0, a0, variables0, assignment0 =
                max_edge_enforcement_no_fixing_linear_program(G)
            var_index0 = {v: i for i, v in enumerate(variables0)}

            for fixed_type, fixed_value in fixed_constraints:
                if fixed_type == "a":
                    m0.add_constraint(a0[var_index0[fixed_value[0]]] ==
                        fixed_value[1])
                else:
                    m0.add_constraint(x0[fixed_value[0]] == fixed_value
                        [1])

            m0.add_constraint(a0[var_index0[symbol]] == 0)

            try:
                m0.solve()

                m = m0
                x = x0
                a_lp = a0
                variables = variables0
                assignment = assignment0

                fixed_constraints.append(("a", (symbol, 0)))
                fixed_consistent_sets.add(group_index)

                solution_x = m.get_values(x)
                is_noninteger = any(abs(solution_x[i, j] - round(
                    solution_x[i, j])) > 1e-6 for i in range(n) for j in
                    range(n))

                if not is_noninteger:
                    return m, x

                continue

            except MIPSolverException:
                pass

            m1, x1, a1, variables1, assignment1 =
                max_edge_enforcement_no_fixing_linear_program(G)
            var_index1 = {v: i for i, v in enumerate(variables1)}

            for fixed_type, fixed_value in fixed_constraints:
                if fixed_type == "a":
                    m1.add_constraint(a1[var_index1[fixed_value[0]]] ==
                        fixed_value[1])
```

```
                else:
                    m1.add_constraint(x1[fixed_value[0]] == fixed_value
                        [1])

            m1.add_constraint(a1[var_index1[symbol]] == 1)

            try:
                m1.solve()

                m = m1
                x = x1
                a_lp = a1
                variables = variables1
                assignment = assignment1

                fixed_constraints.append(("a", (symbol, 1)))
                fixed_consistent_sets.add(group_index)

                solution_x = m.get_values(x)
                is_noninteger = any(abs(solution_x[i, j] - round(
                    solution_x[i, j])) > 1e-6 for i in range(n) for j in
                    range(n))

                if not is_noninteger:
                    return m, x

                continue

            except MIPSolverException:
                pass

            fixed_consistent_sets.add(group_index)

            continue

        if available_inconsistent_sets:
            group_index, group = available_inconsistent_sets[0]
            star = choose_inconsistent_star(G, [group])

            if star is None:
                fixed_inconsistent_sets.add(group_index)
                continue

            branch_found = False

            for edge in sorted(star["edges"]):
                value = 1

                m_branch, x_branch, a_lp_branch, variables_branch,
                    assignment_branch =
                    max_edge_enforcement_no_fixing_linear_program(G)
                var_index_branch = {v: i for i, v in enumerate(
                    variables_branch)}

```

```
                for fixed_type, fixed_value in fixed_constraints:
                    if fixed_type == "a":
                        m_branch.add_constraint(a_lp_branch[
                            var_index_branch[fixed_value[0]]] ==
                            fixed_value[1])
                    else:
                        m_branch.add_constraint(x_branch[fixed_value[0]]
                            == fixed_value[1])

                m_branch.add_constraint(x_branch[edge] == 1)

                try:
                    m_branch.solve()

                    m = m_branch
                    x = x_branch
                    a_lp = a_lp_branch
                    variables = variables_branch
                    assignment = assignment_branch

                    fixed_constraints.append(("x", (edge, 1)))
                    fixed_inconsistent_sets.add(group_index)

                    solution_x = m.get_values(x)
                    is_noninteger = any(abs(solution_x[i, j] - round(
                        solution_x[i, j])) > 1e-6 for i in range(n) for j
                        in range(n))

                    if not is_noninteger:
                        return m, x

                    branch_found = True
                    break

                except MIPSolverException:
                    continue

            if not branch_found:
                fixed_inconsistent_sets.add(group_index)

            continue

        return m, x
```

Listing 9: Max Rounding Linear Program.

```
def integer_components_and_ranked_vertex_edges(x_val, n, eps=1e-6):
    G_int = Graph()
    G_int.add_vertices(range(n))

    for (i, j), v in x_val.items():
        if i < j and v > 1 - eps:
            G_int.add_edge(i, j)

```

```
    components = G_int.connected_components()

    comp_index = {}
    for idx, C in enumerate(components):
        for v in C:
            comp_index[v] = idx

    vertex_score = {}
    for (i, j), v in x_val.items():
        if i >= j:
            continue
        if eps < v < 1 - eps:
            w = abs(v - 0.5)
            vertex_score[i] = vertex_score.get(i, 0) + w
            vertex_score[j] = vertex_score.get(j, 0) + w

    if not vertex_score:
        return components, comp_index, None, []

    v_star = max(vertex_score.items(), key=lambda t: (t[1], -t[0]))[0]

    incident_edges = []
    for (i, j), v in x_val.items():
        if eps < v < 1 - eps:
            if i == v_star or j == v_star:
                a, b = min(i, j), max(i, j)
                incident_edges.append((a, b, v, abs(v - 0.5)))

    incident_edges.sort(key=lambda t: (-t[3], t[0], t[1]))

    return components, comp_index, v_star, incident_edges

def max_rounding_linear_program(G, threshold=0.5):
    fixed_assignments = []
    n = G.order()

    m, x = max_linear_program(G)
    m.solve()

    for step in range(n * (n - 1) // 2):
        x_val = m.get_values(x)
        _, comp_index, v_star, edges =
            integer_components_and_ranked_vertex_edges(x_val, n)

        if v_star is None:
            return m, x

        i, j, v, _ = edges[0]
        fix_val = 1 if comp_index[i] != comp_index[j] else (1 if v >
            threshold else 0)

        m_try, x_try = max_linear_program(G)
        for a, b, val in fixed_assignments:
            m_try.add_constraint(x_try[a,b] == val)
```

```
            m_try.add_constraint(x_try[b,a] == val)
        m_try.add_constraint(x_try[i,j] == fix_val)
        m_try.add_constraint(x_try[j,i] == fix_val)

        try:
            m_try.solve()
        except MIPSolverException:
            return m, x

        fixed_assignments.append((i, j, fix_val))
        m, x = m_try, x_try

    return m, x
```

Listing 10: Unicliqual Rounding Linear Program.

```
def unicliqual_rounding_linear_program(G, threshold=0.5):
    fixed_assignments = []
    n = G.order()

    m, x = unicliqual_vertex_linear_program(G)
    m.solve()

    for step in range(n * (n - 1) // 2):
        x_val = m.get_values(x)
        _, comp_index, v_star, edges =
            integer_components_and_ranked_vertex_edges(x_val, n)

        if v_star is None:
            return m, x

        i, j, v, _ = edges[0]
        fix_val = 1 if comp_index[i] != comp_index[j] else (1 if v >
            threshold else 0)

        m_try, x_try = unicliqual_vertex_linear_program(G)
        for a, b, val in fixed_assignments:
            m_try.add_constraint(x_try[a,b] == val)
            m_try.add_constraint(x_try[b,a] == val)
        m_try.add_constraint(x_try[i,j] == fix_val)
        m_try.add_constraint(x_try[j,i] == fix_val)

        try:
            m_try.solve()
        except MIPSolverException:
            return m, x

        fixed_assignments.append((i, j, fix_val))
        m, x = m_try, x_try

    return m, x
```